

ACKNOKLEDGEMENT

At first, we would like to say a special thank you to all Bach Khoa university lecturers in Ho Chi Minh city, who has taught us helpful knowledge with high enthusiasm and concentration. The knowledge we have acquired in the period of 4 years are extremely precious and helpful for our final project.

Furthermore, we want to give our deepest thanks to instructor, Doctor Pham Cong Bang, for his invaluable time and patience to guide us and give us feedback through the studying process. His opinions and ideas on this project have also made us better at what we do.

Besides, we appreciate friends, lab mates, colleagues for the fun time we had working and socializing together.

Last but not least, we value the love and dedicated encouragement of our extended family. Words cannot express our gratitude to you all.

Finally, due to the limit of our knowledge, time and experience, mistakes in our thesis proposal could be overlooked. We are looking forward for your opinion and feedback dear lecturers. There will be the useful experience for us on our career path. Our sincere thanks to you all.

Ho Chi Minh city, January 10th, 2023

Group of Students

Bui Viet Thang

Phan Tien Minh

ABSTRACT

Soccer robot is a specialized autonomous robot and mobile robot that is used to play variants of soccer system, Soccer robot competition designed to advance robotics and Artificial Intelligence research through friendly competition. The main organized competitions are RoboCup or FIRA tournaments played each year. Based on the existing conceptual design, this thesis will improve and develop differential wheeled soccer robot in practice.

This project includes 6 chapters as follows:

CHAPTER 1: INTRODUCTION – The overview about the soccer robot competitions and leagues.

CHAPTER 2: MECHANICAL DESIGN – The calculation and selection of mechanical components for the mechanical components.

CHAPTER 3: ELECTRICAL AND CONTROLLER DESIGN – Design the electrical system and PID controller.

CHAPTER 4: COMPUTER VISION SYSTEM – Recognize ball and robot by image processing.

CHAPTER 5: ROBOT CONTROL ALGORITHM – Generate virtual path for the robot to follow

CHAPTER 6: CONCLUSION AND RECOMMENDATION – Evaluate the system and propose plans for future improvement.

TABLE OF CONTENT

ACKNOKLEDGEMENT	i
ABSTRACT	ii
TABLE OF CONTENT	iii
TABLE OF FIGURES	vi
TABLE OF TABLES	ix
CHAPTER 1: OVERVIEW.....	1
1.1 Introduction.....	1
1.2 Soccer robots and teams.....	2
1.2.1 RoboCup Soccer Humanoid League.....	2
1.2.2 RoboCup Standard Platform League (NAO).....	3
1.2.3 RoboCup Middle Size Robot League	3
1.2.4 RoboCup Junior League	4
1.3 IEEE Very Small Sized Soccer (VSSS)	5
1.4 Robot Soccer System by a group of K17 OISP mechatronics students	7
1.4.1 Overview	7
1.4.2 Prototype development	8
1.4.3 Experiment result	9
1.5 Objective and Scope	9
CHAPTER 2: MECHANICAL DESIGN.....	12
2.1 Select robot's wheels	13
2.2 Select an electric motor.....	13
2.3 Design gear transmission	16
2.3.1 Design data.....	16
2.3.2 Choosing material	16
2.3.3 Calculation for gears	17
2.3.3 Gear drive parameters	18
2.3.4 Strength check.....	19
2.4 Design shaft	22
2.5 Select bearing.....	23
2.6 Evaluation	25
CHAPTER 3: ELECTRICAL AND CONTROLLER DESIGN	26
3.1 Driver and Motor module	26

3.2 Microcontroller module	28
3.3 Buck converter	30
3.4 Power module	31
3.5 Printed Circuit Board (PCB).....	33
3.6 Mathematic model	34
3.7 PID controller design	36
3.8 Evaluation	40
3.8.1 Electrical hardware	40
3.8.2 PID controller.....	41
3.9 Conclusion	42
CHAPTER 4: COMPUTER VISION.....	44
4.1 Selecting camera	44
4.2 Camera installation	45
4.3 Image processing	46
4.3.1 Image processing procedure	46
4.3.2 Coordinate calibration.....	46
4.4 Robot recognition.....	47
4.5 Ball recognition.....	48
4.5.1 Color detection.....	49
4.5.2 Circle detection	49
4.6 Flowchart	50
4.7 Evaluation	52
4.8 Conclusion	54
CHAPTER 5: CONTROL ALGORITHM	56
5.1 The criteria to design the control algorithm.....	56
5.2 General workflow	56
5.3 JSON	57
5.4 Strategy to shoot a ball.....	57
5.4.1 Path planning	58
5.4.2 A* algorithm	59
5.4.3 Heuristics	61
5.4.4 Evaluation	63
5.5 Socket network.....	64
5.5.1 The client-server model	65

5.5.2 ESP 32 Wi-Fi station mode:	66
5.5.3 Client programing method:	67
5.6 Robot's main controller	69
5.7 Sub program localization	71
5.8 Sub program state control	71
5.9 Evaluation	74
5.10 Conclusion	76
CHAPTER 6: CONCLUSION AND SUMMARY	77
6.1 Summary of achievement	77
6.2 Summary of limitation	77
6.3 Evolution method suggestion.....	77
References.....	79

TABLE OF FIGURES

Figure 1.1 Standard form robot compete in soccer match the RoboCup 2019 in Graz, Australia [2]	1
Figure 1.2 Team NimbRo, winner of the Best Humanoid Award 2019[4]	2
Figure 1.3 A NAO and official ball [5].....	3
Figure 1.4 RoboCup MSL Leipzig, Germany (2016) [7]	4
Figure 1.5 Two teams of two robots with an orange ball on a RoboCup Junior Soccer field [8]	4
Figure 1.6 IEEE Very Small Size RoboCup Soccer in the Latin American and Brazilian Robotics Competition (LARC/CBR) [10]	5
Figure 1.7 Arena dimensions [10].....	6
Figure 1.8 The 24th International Conference on Mechatronics Technology (ICMN 2021) will be held online by Singapore from December 18-22, 2021.	7
Figure 1.9 Final design of the robot.....	8
Figure 1.10 Two soccer robots and the ball [11]	8
Figure 1.11 Dimension of a soccer field.....	10
Figure 2.1 Soccer robot mechanism	12
Figure 2.2 Schematic diagram of soccer robot	12
Figure 2.3 Force analysis diagram	14
Figure 2.4 GA25 gear motor DC with speed encoder	16
Figure 2.5 Shaft moment diagram	23
Figure 2.6 Mechanical design result	25
Figure 3.1 Electrical principal diagram.....	26
Figure 3.2 Motor controller block.....	28
Figure 3.3 TB6612FNG driver [13]	27
Figure 3.4 ESP32 DEVKIT V1 pinout diagram [14]	29
Figure 3.5 Controller block	30
Figure 3.6 LM2596 buck converter [15]	31
Figure 3.7 Lithium Li-on pin case	32
Figure 3.8 Lithium Li-on pin	33
Figure 3.9 Power module.....	33
Figure 3.10 Printed circuit board (PCB)	34

Figure 3.11 Kinematic model	35
Figure 3.12 Error model.....	36
Figure 3.13 Revolution per minute correspond to %PWM (Right motor)	37
Figure 3.14 Revolution per minute correspond to %PWM (Left motor).....	38
Figure 3.15 Left motor response PID	39
Figure 3.17 Final printed circuit board (PCB)	40
Figure 3.18 Counting pulses process.....	41
Figure 3.19 Recheck the distance	42
Figure 4.1 C922 Pro HD stream webcam	44
Figure 4.2 Camera installation model.....	45
Figure 4.3 Image processing workflow	46
Figure 4.4 Expected Coordinates after Calibration	47
Figure 4.5 ArUco marker used in this project.....	47
Figure 4.6 Marker detection workflow	48
Figure 4.7 Inesis golf ball.....	48
Figure 4.9 Color detection result.....	49
Figure 4.11 Final result of recognizing the ball.....	49
Figure 4.12 Flow chart of sub program	50
Figure 4.13 Final computer vision flowchart	51
Figure 4.14 Ball positions.....	52
Figure 4.15 Robot positions.....	53
Figure 4.16 Chosen testing location.....	53
Figure 5.1 General workflow of soccer robot	57
Figure 5.2 Types of SLAM[17]	58
Figure 5.3 Path planning objectives of the soccer robot project	61
Figure 5.4 A* flowchart.....	61
Figure 5.5 Manhattan Distance heuristics	62
Figure 5.6 Diagonal Distance heuristics.....	62
Figure 5.7 Euclidean Distance heuristics.....	63
Figure 5.8 Path planning simulation	63
Figure 5.9 Simulation to dodge enemy	64
Figure 5.10 ESP 32 board act as Wi-Fi Station [20]	67

Figure 5.11 ESP 32 connect to Wi-Fi and show IP address	67
Figure 5.12 Working principle of ESP 32 and PC client program interact each other	68
Figure 5.13 Main program flowchart.....	70
Figure 5.14 The flowchart of state machine	72
Figure 5.15 Difficult positions	74
Figure 5.16 20 Test cases design to evaluate the performance of the robot (in order from left to right and up to down)	76

TABLE OF TABLES

Table 2.1 Desired parameter	14
Table 2.2 Motor parameter	15
Table 2.3 Gear drive parameters	19
Table 3.1 TB6612FNG driver datasheet	28
Table 3.2 Esp32 datasheet [22]	29
Table 3.3 LM2596 buck module parameter	31
Table 3.5 Total power of the electrical system	31
Table 3.6 Lithium Li-on battery parameters	32
Table 3.7 Revolution per minute correspond to %PWM (Right motor)	37
Table 3.8 Revolution per minute correspond to %PWM (Left motor)	38
Table 3.9 Set of PID parameters	39
Table 3.10 Counting rising pulses record	41
Table 3.11 Recheck the distance result	42
Table 4.1 Technical Specification C922 Pro HD stream webcam	45
Table 4.2 Ball recognition error result (first method)	52
Table 4.3 Robot recognition error result (first method)	53
Table 4.4 Ball recognition error result (second method)	54
Table 5.1 Difference between TCP and UDP protocol [19]	66
Table 5.2 Probability of goal	74
Table 5.3 The table of the performance of the robot over 20 test cases	75

CHAPTER 1: OVERVIEW

1.1 Introduction

In recent years, using mobile robots in the technical, industrial, and educational applications has been increased. Robotics is gaining popularity among younger generation, which is quite evident by the tremendous increase in robotic competitions all over the world.

Soccer also football nowadays becomes the most favorable sport in the world. Nothing unusual if robot soccer becomes more popular robot competition over the last decade. In 1993, Alan Mackworth proposed the idea of robot soccer game, which promotes the full integration of the research into artificial intelligence and robotics. There are some international soccer robot organizations who divide the competitions into several leagues, each of these leagues focus on the different technologies. Nowadays, there are 2 main international associations: RoboCup and FIRA.[1]

RoboCup is an annual international robotics competition proposed and founded in 1996 by a group of university professors (including Hiroaki Kitano, Manuela M. Veloso, and Minoru Asada). The aim of the competition is to promote robotics and AI research by offering a publicity appealing – but formidable – challenge.



Figure 1.1 Standard form robot compete in soccer match the RoboCup 2019 in Graz, Australia [2]

The focus of the Robo Cup competition is the game of football/soccer, where the research goals concern cooperative multi-robot and multi-agent systems in dynamic adversarial environments. At this time, RoboCup's leagues has divided into 5 leagues: Humanoid, Standard Platform, Middle Size, Small Size and Simulation. Figure 1.1

shows the NAO robot from SoftBank Robotics (The RoboCup Standard Platform League).

1.2 Soccer robots and teams

1.2.1 RoboCup Soccer Humanoid League

Robots participating in the Humanoid League competitions must have a human-like body plan and human-like senses play soccer against each other. They must consist of two legs, two arms, and one head, which are attached to a trunk. Unlike humanoid robots outside the Humanoid League the task of perception and world modeling is not simplified by using non-human like range sensors. In addition to soccer competitions technical challenges take place. Dynamic walking, running, and kicking the ball while maintaining balance, visual perception of the ball, other players, and the field, self-localization, and team play are among the many research issues investigated in the Humanoid League [3]

Robot must act autonomously while a competition is running. No external power supply, teleoperation, remote control, or remote brain of any kind is allowed. Robots may communicate only via the (wireless) network provided by the organizers, which must support the referee box.



Figure 1.2 Team NimbRo, winner of the Best Humanoid Award 2019[4]

1.2.2 RoboCup Standard Platform League (NAO)

Standard Platform League is a RoboCup robot soccer league, in which all teams compete with identical robots. The robots operate fully autonomously, there is no external control, neither by humans nor by computers. The teams play on a green field with white lines and goal posts, with no other landmarks, and the ball consists in a realistic white and black soccer one. These game characteristics generate a very challenging scenario, which allows improving the league every year.

All teams must use black, gray, red, blue, or orange plated NAO humanoid robots manufactured by SoftBank Robotics. No additional hardware is permitted including off-board sensing or processing systems.[5]

The robot can communicate in 2 ways: Wireless communications and non-wireless communications.



Figure 1.3 A NAO and official ball [6]

1.2.3 RoboCup Middle Size Robot League

In RoboCup MSL teams of 5 fully autonomous play soccer with a regular size FIFA soccer ball. Teams are free to design their own hardware but all sensors have to be on-board and there is a maximum size and weight for the robots. The research focus is on mechatronics design, control and multi-agent cooperation at plan and perception levels. [7]

The projection of the robot's shape onto the floor fits into a square of size at least 30cm × 30cm and at most 52cm × 52cm. Communication between the robots of a team using wireless. Communication between the robots and one remote computer system

(herein after referred as Base Station) is also allowed.



Figure 1.4 RoboCup MSL Leipzig, Germany (2016) [7]

1.2.4 RoboCup Junior League

Robots must be constructed and programmed in a way that their movement is not limited to only one dimension (defined as a single axis, such as only moving in a straight line). They must move in all directions, for example by turning. The size of the robot is 4cm diameter round.

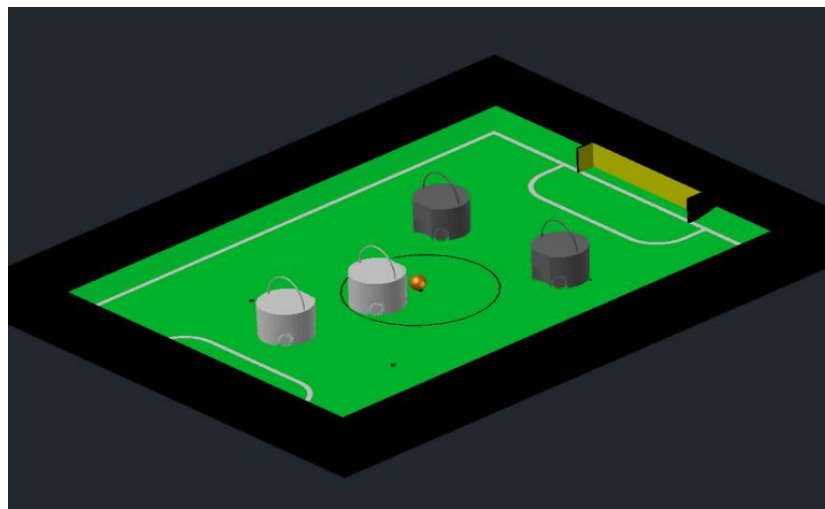


Figure 1.5 Two teams of two robots with an orange ball on a RoboCup Junior Soccer field [8]

Robots are not allowed to use any kind of communication during gameplay unless the communication between robots is via Bluetooth class 2 or class 3 or via any other device that communicates using the 802.15.4 protocol (e.g., ZigBee and XBee).

1.3 IEEE Very Small Sized Soccer (VSSS)

The IEEE Very Small Size Soccer (VSSS) is a robot soccer competition in which two teams of three small robots play against each other. During the game, the main task for each team is to navigate on the field, moving the ball to score goals. Once the match starts, the robots cannot receive any human interaction, except during the break time. An image of the VSSS field is shown in Figure 1.6.



Figure 1.6 IEEE Very Small Size RoboCup Soccer in the Latin American and Brazilian Robotics Competition (LARC/CBR) [9]

1.3.1 The field and the ball

A black (non-reflective) rectangular wooden playground, 150cm x 130cm in size with 5cm high and 2.5 thick with sidewall will be used. The topsides of the sidewalls shall be black in color with the walls painted in white (side view). Solid 7cm X 7cm isosceles triangles shall be fixed at the four corners of the playground to avoid the ball getting cornered. The surface texture of the board will be that of a ping pong table.

The field of play shall be marked as shown in Figure 1.7. The center circle will have a radius of 20cm. The arc, which is part of the goal area, will be 20cm along the goal line and 5cm perpendicular to it. The major lines/arcs (centerline, goal area borderlines and the center circle) will be white in color and 3mm in thick. The free ball (Law 13) robot positions (circles) shall be marked in gray or white color.

The goal shall be 40cm wide with 10cm of depth. Posts and nets shall not be provided at the goal. This 10cm go beyond the 150cm of the field. The goal line is the line just in front of the goal which is 40cm long. The goal areas shall comprise of areas contained by the rectangle (sized 70cm X 15cm in front of the goal) and the attached

arc (20cm in parallel to the goal line and 5cm perpendicular to it).

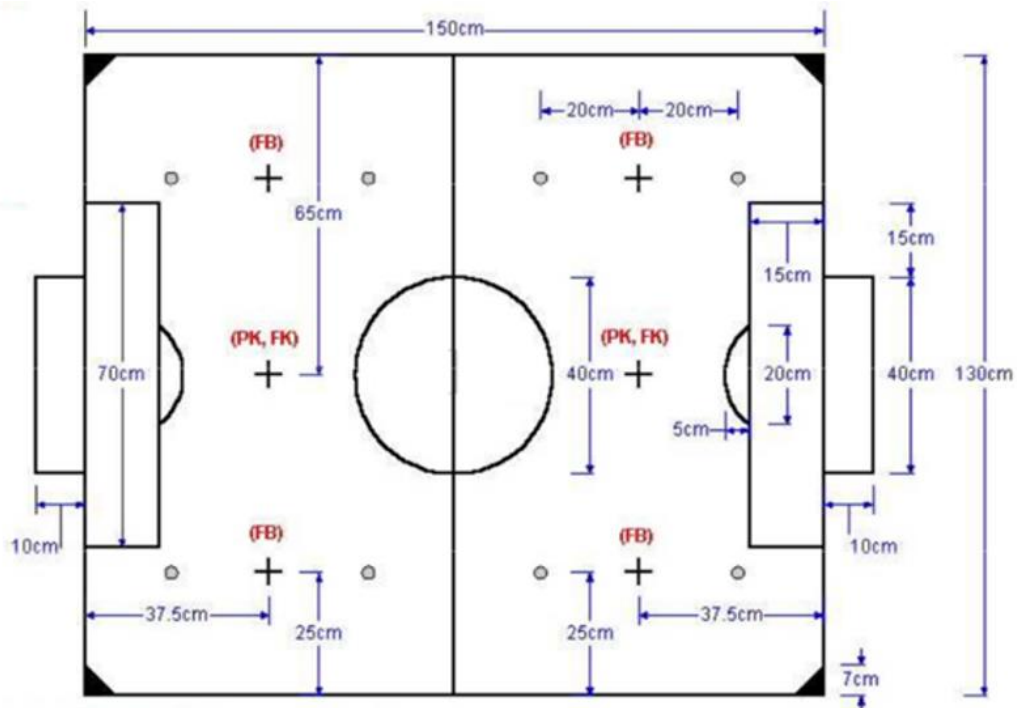


Figure 1.7 Arena dimensions [10]

An orange golf ball shall be used as the ball, with 42.7mm diameter and 46g weight. To avoid vision problems, the ball will be provided by the competition. The field shall be indoors. The lighting condition in the competition site shall be fixed around 1000 Lux.

1.3.2 The players

A match shall be played by two teams, each consisting of a maximum of three robots. One of the robots can be the goalkeeper. Three human team members, a” manager”, a” coach” and a” trainer” shall only be allowed on stage. One host computer per team, mainly dedicated to vision processing and for location identifying, can be used.

The size of each robot shall be limited to 7.5cm x 7.5cm x 7.5cm. The height of the RF communication antenna will not be considered in deciding a robot's size. The robots should wear uniforms and the size of which shall be limited to 8cm x 8cm x 8cm. A uniform cannot be part of the robot, i.e., the robot must be able to play a game even without the uniform.

The topside of a robot must not be colored in orange, white or gray nor must not be

Chapter 1: Overview

colored with more than two colors different from black and the team color. A color patch either blue or yellow, as assigned by the organizers, will identify the robots in a team. All the robots must have, visible on their top, a solid region of their team color patch, blue or yellow. This region can be of any shape but must be able to contain (at least) a square with 3.5 cm of side or a circle with 4 cm of diameter. A team's identification color will change from game to game, and the team color patch used should be detachable. When assigned with one of the 2-team colors (blue or yellow), the robots must not have any visible patches of those colors used by an opponent team. Note: The teams are recommended to prepare a minimum of double blue and yellow patches for competition. The teams are also recommended to prepare a minimum of 6 different color patches, other than blue and yellow, for individual robot identification.

1.4 Robot Soccer System by a group of K17 OISP mechatronics students

1.4.1 Overview

A study “Study and Implement of a Robot Soccer System based on the CDIO approach” begun developed by a group of K2017 OISP mechatronics students (Tran Tieu Binh – team leader, Cao Tri Trieu, Dang Minh Quan and Lam Hung Minh). After six months of research and experimentation in the context of the COVID-19 epidemic in Ho Chi Minh City, the team sometimes ran the model directly at the school, sometimes had to write code remotely due to the global social distance in the city.



Figure 1.8 The 24th International Conference on Mechatronics Technology (ICMN 2021) will be held online by Singapore from December 18-22, 2021.

Overcoming many obstacles with expert guidance from Dr. Pham Cong Bang - Lecturer in Mechatronics Department, a group of OISP students completed “Study and

Implement of a Robot Soccer System based on the CDIO approach” (thesis title) and was accepted to participate in the project. attend the 24th International Conference on Mechatronic Technology (ICMT 2021) hosted by Singapore online from December 18-22, 2021 showed in fig. 1.8.

1.4.2 Prototype development

The robot is required to fit into a $100 \times 100 \times 100$ (mm) space, driven by two wheels and the robot’s structure must be modular. In this study, the soccer robot is a differential drive mobile platform. This structure consists of two driving wheels mounted on a common axis, and each wheel can independently be driven either forward or backward. The center wheel axle is a good design choice, the robot rotates around its center, which supports robot detection later in vision. The robot could be well-balanced by adding two small ball caster wheels on the two sides of the axle. The robot soccer separating in four layers as highlighted in the figure 1.9.

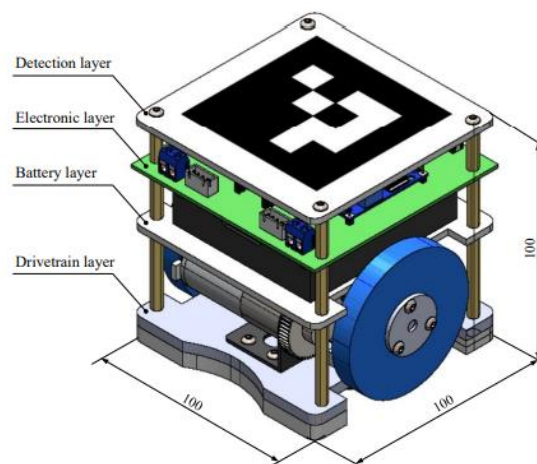


Figure 1.9 Final design of the robot

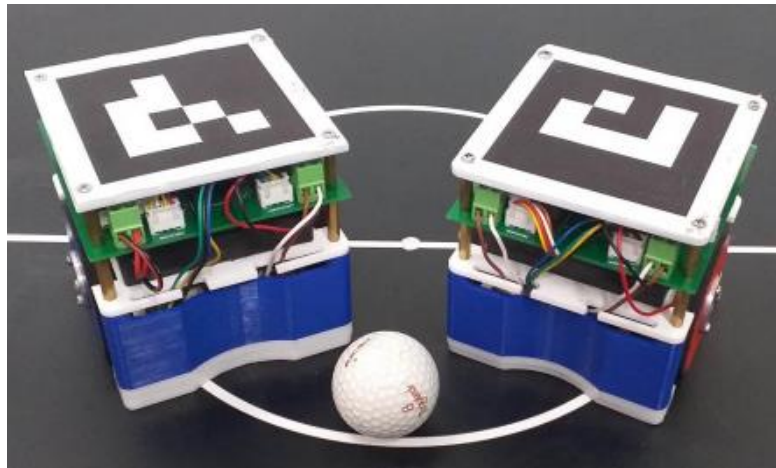


Figure 1.10 Two soccer robots and the ball [11]

1.4.3 Experiment result

From the design above, two robots are created as in Fig. 1.10. When applying the control method, these robots can achieve the accuracy of $\pm 10^\circ$ for pure rotation and ± 50 mm for pure translation. The vision-based robot localization, which makes sure the system works as desired, has been deployed. The experimental results with respect to time illustrates the success of the controller system. However, the accuracy performance of the system is not very good, which definitely limits the robot ability in team coordination. Therefore, together with the simultaneous motion between the rotation and translation that makes the robot move smoothly, the improvement in system performance will be our future works.

1.5 Objective and Scope

1.5.1 Goal

Based on foreign projects and previous project made by Tran Tieu Binh and his friends, the main objective in this thesis is implementing the prototype and kinematic of the robot, computer vision algorithms, object recognition and motion control algorithm of the robot to improve the movement of the previous soccer robot system.

1.5.2 Objective

a) Calculate and Design soccer robot system

- Design half of the soccer field (80cm x 100cm)
- Design computer vision system
- Implement and design a differential wheeled robot
- Design electrical circuit
- Calculate transfer function and PID control system for the motor
- Experiment and evaluate the accurate of the control system based on the response of the robot

b) Computer vision for robot and ball localization

- Research the algorithm like Hough Circle, Canny Edge detection, Aruco detection to calculate position of the robot and the ball
- Implement and design flowchart and write code program for computer vision

system

c) Develop the motion control system

- Research Design the communication between the robot and PC via WIFI
- Research path planning algorithm to shoot the ball into the goal
- Experiment the robot to shoot a ball

1.5.3 Thesis scope

- Within the objective, the scope is limited for: The half soccer field have a dimension of 1000mm x 800mm, flat surface made of PVC plastic. Figure 1.11 is shown the dimension of half the soccer field for the robot.
- A colored golf ball shall be used as the ball, with standard weight and diameter.
- The robot prototype is differential wheel robot with 2 main motor and 1 free turning wheel.

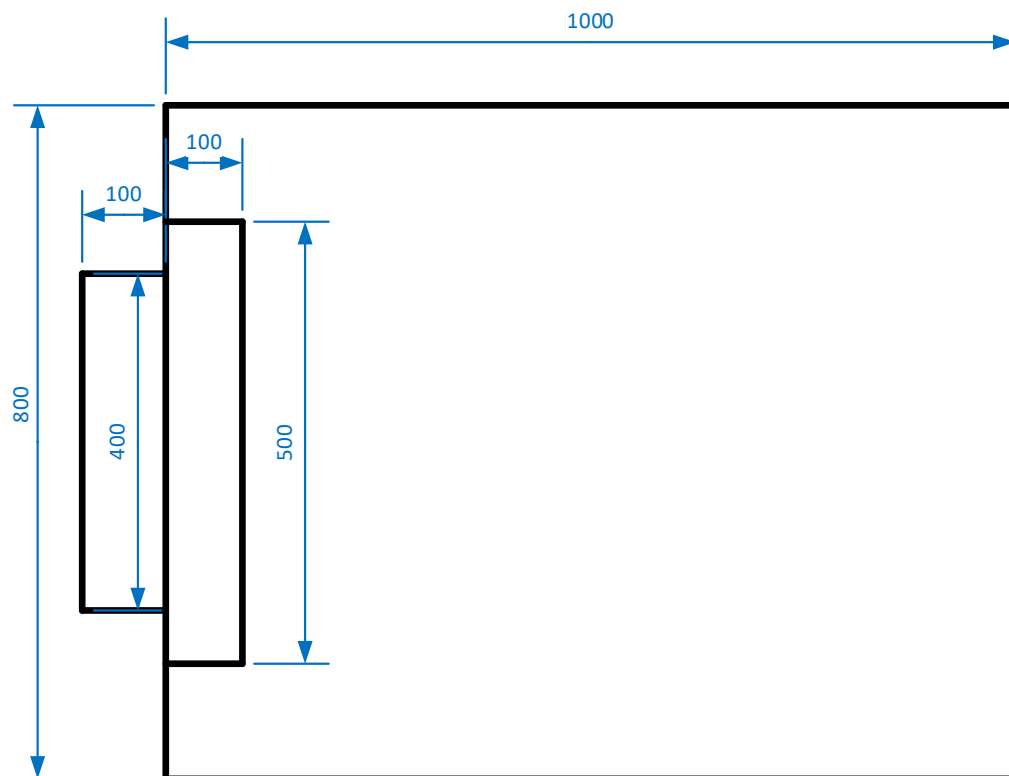


Figure 1.11 Dimension of a soccer field

- A colored golf ball shall be used as the ball, with standard weight and diameter.
- The robot prototype is differential wheel robot with 2 main motor and 1 free turning wheel.

1.5.4 Thesis's chapters

- Chapter 2: Design mechanical component for the robot
- Chapter 3: Design electrical part for the robot
- Chapter 4: Implement computer vision solutions for the robot
- Chapter 5: Path-planning for the robot
- Chapter 6: Evaluate the project

CHAPTER 2: MECHANICAL DESIGN

The purpose of this chapter is to fabricate a small size soccer robot (100 x 100 x 100 mm) and light weight. Therefore, the robot platform consist of 2 driving wheels at the rear and one ball wheel in the front is selected. The ball wheel is suitable with our flat plane platform as it provides 1 contact point, which produce less friction compared to other type of wheel, reducing the occurrence of slipping that produce noise for the control algorithm. Figure 2.1 illustrates the soccer robot mechanism.

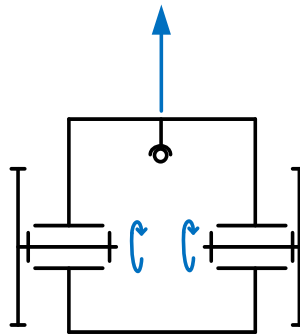


Figure 2.1 Soccer robot mechanism

The mechanism is no need to implement self-balanced algorithm. However, the passive wheel might cause undesired movement, increasing the complexity of kinematic and dynamic analysis. Moreover, because the robot uses 2 motors to control wheels, therefore the installation space problem raises. Hence, the gear transmission is not only to increase the robot speed, but it also allows 2 motors installed on a square with area of 100 mm^2 . Figure 2.2 describes the schematic diagram of soccer robot.

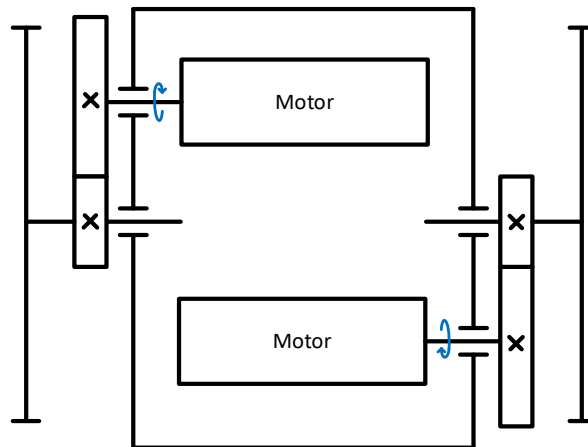


Figure 2.2 Schematic diagram of soccer robot

Chapter 2: Mechanical design

Based on the scope, the design parameters need to be followed as below

- Maximum size: 100x100x100 mm
- Mass = 1kg
- Velocity: up to 0.15m/s

2.1 Select robot's wheels

Based on the soccer robot mechanism, the wheels are designed and fabricated with following parameters:

- Wheel diameter: $d_a = 86 \text{ mm}$.
- Wheel width: $w_a = 5 \text{ mm}$.

For the passive wheel, select small universal castor wheel with the specifications:

- Wheel width: $d_b = 22 \text{ mm}$.
- Wheel height: $h_b = 15 \text{ mm}$.

2.2 Select an electric motor

Figure 2.3 illustrates the force analysis diagram on the wheel, where R is wheel's radius, $\vec{P}$ is gravitational force on wheel by the Earth, $\vec{v}$ is wheel's velocity direction, $\vec{N}$ is contact normal force on wheel by floor and $\vec{F}_{ms}$ is frictional force

* Moment of inertia of the wheel:

$$I = \frac{1}{2}mR^2 \quad (2.1)$$

* Equalize the moment of inertia around the center of the wheel:

$$\tau - F_{ms}R = I\gamma \quad (2.2)$$

$$\leftrightarrow \tau = I\gamma + F_{ms}R \quad (2.3)$$

* Condition that the wheel is rolling without slipping when motor shape rotating:

$$F_{ms} \leq \mu N = \mu P \quad (2.4)$$

$$\leftrightarrow \tau \leq \mu PR + I\gamma \quad (2.5)$$

$$\leftrightarrow \tau \leq \frac{1}{2}mR^2\gamma + \mu mg \times R \quad (2.6)$$

* The power of the wheel:

$$P_{\text{wheel}} = \tau\omega \quad (2.7)$$

* Required power of motor:

$$P_{motor} = \frac{P_{wheel}}{\eta} \quad (2.8)$$

In which:

- $m = 1 \text{ kg}$ Total mass
- $\mu = 0,05$ Coefficient of friction
- $R = 43 \text{ mm}$ Wheel radius
- F_{ms} Friction force
- $\gamma = a/R$ Angular acceleration
- $a = 0.1 \text{ m/s}^2$ Desired acceleration
- $g = 9,81 \text{ m/s}^2$ Gravitational acceleration
- ω Angular velocity
- η Transmission efficiency

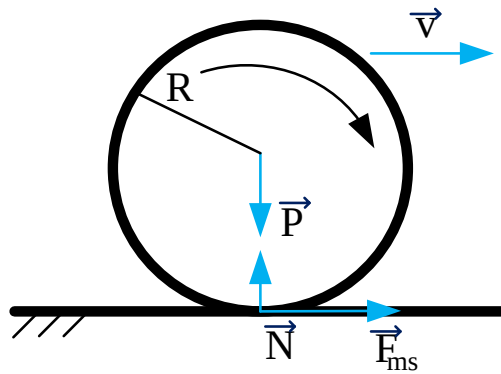


Figure 2.3 Force analysis diagram

Table 2.1 Desired parameter

Parameter	Value
Desired velocity	0.1 (m/s)
Wheel radius	43 (mm)
Friction coefficient	0.05
Safety coefficient	1.3

Hence, the moment of inertia of the working shaft:

$$\begin{aligned} \tau &= \frac{1}{2} \times 1 \times (43 \times 10^{-3})^2 \cdot \frac{1}{43 \cdot 10^{-3}} + 0,05 \times 1 \times 9.81 \times 43 \times 10^{-3} \\ &= 0.04 \text{ Nm} \end{aligned}$$

Chapter 2: Mechanical design

Number of revolutions of the working shaft:

$$n_{shaft} = \frac{60v}{\pi D} = \frac{60 \times 1}{\pi \times 86.10^{-3}} = 238.8 \text{ m/s} \quad (2.9)$$

The transmission ratio of spur gear is 0.5, hence minimum motor rotation speed is:

$$n_{motor} = u_r \times n_{shaft} = 0.5 \times 238.8 = 119.4 \text{ m/s}$$

The transmission system consists of 1 pair of spur gears, 1 pair of bearings. Hence the efficiency of the system can be calculated as below:

$$\eta = \eta_{spurgear} \eta_{bearing} \quad (2.9 \text{ p19[12]})$$

where:

$$\eta_{spurgear} = 0.94: \text{Efficiency of a pair of spur gears}$$

$$\eta_{bearing} = 0.95 : \text{Efficiency of a pair of bearings}$$

The efficiencies of gears and bearing are chosen from table 2.3 p.19[12], given that the gears not covered, it is suggested that choosing the value in a specific range; given no practical experiences, all the middle values from those ranges are chosen for better accuracy.

$$\Rightarrow \eta = 0.94 \times 0.95 = 0.83$$

Choose safety coefficient of motor: $s_f = 1.3$

Moment of inertia of the motor shaft:

$$T = \frac{\tau s_f}{\eta u_r} = \frac{0.04 \times 1.3}{0.83 \times 0.5} = 0.125 \text{ Nm} \quad (2.10)$$

Choose GA25 gear motor DC with speed encoder. Figure 2.4 illustrates the chosen motor.

Table 2.2 Motor parameter

Parameter	Value
mass	0.1 kg
Voltage	3 - 12 V
Motor speed	200 rpm
Moment of inertia	420 Nmm
Power	13.2 W



Figure 2.4 GA25 gear motor DC with speed encoder

Recalculate the system:

Average speed of robot:

$$v_{ct} = \frac{(n_{ct}\pi D)}{60} = \frac{200 \times \pi \times 84.10^{-3}}{60} = 0.015 \frac{m}{s} \quad (2.11)$$

Power of motor shaft:

$$P_{ct} = \frac{T \cdot \pi \cdot n_{ct}}{30} = \frac{0.125 \times \pi \times 200}{30} = 2.616 W < P_{dm} = 13.2 W \quad (2.12)$$

Hence, the chosen motor satisfies the requirement

Power of working shaft:

$$P_{wheel} = P_{ct} \times \eta = 0.942 \times 0.83 = 0.78 W$$

Torque of working shaft:

$$T_{shaft} = \frac{60 \times P_{wheel}}{2\pi \times n} = 0.037 Nm$$

2.3 Design gear transmission

2.3.1 Design data

- Power of helical gear: $P = 9.63 W$
- Transmission ratio of helical gear: $u = 0.5$
- Revolution of driving gear: $n_1 = 200 rpm$
- Revolution of driven gear: $n_2 = 400 rpm$

2.3.2 Choosing material

According to table 6.1 p.92[12], without any specific requirement, choose widely

Chapter 2: Mechanical design

used material type I material ($HB \leq 350$), to facilitate the gear's teeth cutting procedure, make it more accurate, lessen the need for stronger material for shafts, but also provide high allowable stress; $H_1 \geq H_2 + (10 \dots 15) HB$ so as to reduce the friction between gears:

For the driving gear we choose steel 45 quenched with rigidity $H_1 = 185HB$; ultimate strength $\sigma_b = 650 MPa$; yield limit $\sigma_{ch} = 380 MPa$

For the driven gear we choose steel 45 quenched with rigidity $H_2 = 170HB$; ultimate strength $\sigma_b = 600 MPa$; yield limit $\sigma_{ch} = 340 MPa$

2.3.3 Calculation for gears

Select the coefficient of the tooth width:

According to table 6.6 p97[12] choose $\psi_{ba} = 0.313$ (the two gears are symmetrical to the axis and $H_1, H_2 < 350HB$)

Select the initial center distance of the axes:

$$a_w = K_a(u + 1) \sqrt[3]{\frac{T_1 K_{H\beta}}{\psi_{ba} [\sigma_H]^2 u}} = 21.6 \text{ mm} \quad (6.15 \text{ p96[12]})$$

Where $K_a = 43$ is the material coefficient chosen from table 6.5 p.96[12]

$T_1 = 65607$ is the torque of the driving gear

$K_{H\beta} = 1$ is the coefficient for undistributed load on gear

$\psi_{ba} = 0.25$ is coefficient of symmetrical

$[\sigma_H] = 386.4$ (MPa) is the allowable tangential stress

Based on SEV229-75 standard, choose $a_w = 25\text{mm}$, which satisfy the condition: closet to the calculated a_w

Gear module:

$$m = (0.01 \div 0.02) a_w = (0.01 \div 0.02) \times 25 = 0.3 \div 0.5$$

Choose $m = 1$

Driving gear teeth:

$$z_1 = \frac{2a_w}{m(u + 1)} = 33.333$$

Choose $z_1 = 34$

Driven gear teeth: $z_2 = z_1 u = 16.666$, choose $z_2 = 17$

Width of driving gear: $b_{w1} = \psi_{ba} a_w = 6.3\text{mm}$

Choose width for driven gear larger than driving gear about 1mm for better meshing

Geometry parameters of the gears: (Equations in table 6.11 p.104[12])

- a) Pitch circle diameter of driving gear:

$$d_1 = z_1 m = 34mm$$

- b) Pitch circle diameter of driven gear:

$$d_2 = z_2 m = 17mm$$

Choose $x_1, x_2 = 0$ (the shifting coefficients) based on table 6.9 p100[12], given that $z_1 = \geq z_{min} + 2 = 18$ ($z_{min} = 16$ corresponding to $\beta = 0^\circ$); $\Delta y = 0$, because displacement is not needed that it might affect other qualities of gear, such as reducing messing efficiency.

- c) Crest circle diameter of driving gear:

$$d_{a1} = d_1 + 2(1 + x_1 - \Delta y)m = 36mm$$

- d) Crest circle diameter of driven gear:

$$d_{a2} = d_2 + 2(1 + x_2 - \Delta y)m = 19mm$$

- e) Pitch center distance:

$$a = 0.5(d_2 + d_1) = 25.5mm$$

- f) Center distance

$$a_w = a + ym = 25.5mm$$

- g) Driving gear reference diameter:

$$d_{w1} = \frac{2a_w}{u + 1} = 34mm$$

- h) Driven gear reference diameter:

$$d_{w2} = d_{w1}u = 17mm$$

- i) Driving gear dedendum diameter:

$$d_{f1} = d_1 - (2.5 - x_1)m = 31.5mm$$

- j) Driven gear dedendum diameter:

$$d_{f2} = d_2 - (2.5 - x_2)m = 14.5mm$$

- k) Driving gear speed:

$$v = \frac{\pi d_{w1} n_1}{60000} = \frac{\pi \times 34 \times 200}{60000} = 0.36m/s$$

According to table 6.13, 6.14 p.107[12], given that $v = 0.36m/s$, choose the degree of accuracy is 9, choose the undistributed load coefficient $K_{H\alpha} = 1.13$, $K_{F\alpha} = 1.37$

2.3.3 Gear drive parameters

All the parameters are summarized in table 2.3.

Table 2.3 Gear drive parameters

Geometry parameters	Spur gear drive	Driving gear	Driven gear
Center distance: a (mm)	25	-	-
Torque: T (Nmm)	-	460	287
Transmission ratio: u	0.5	-	-
n (rpm)	-	200	400
Module: m (mm)	1	-	-
Teeth number: z (teeth)	-	34	17
Teeth angle: β (°)	0	-	-
Pitch circle diameter: d (mm)	-	34	17
Crest circle diameter: d_a (mm)	-	36	19
Dedendum diameter: d_f (mm)	-	31.5	14.5
Gear width: b (mm)	-	6	7

2.3.4 Strength check

The calculated tangential stress:

$$\sigma_H = \frac{Z_M Z_H Z_\varepsilon}{d_1} \sqrt{\frac{2T_1 K_H (u+1)}{b_w u}} \quad (6.58 \text{ p.115[12]})$$

$$= \frac{190 \times 1.76 \times 0.79}{34} \sqrt{\frac{2 \times 460 \times 1.23 \times (0.5 + 1)}{7 \times 0.5}} = 209.7 \text{ MPa}$$

where:

$$Z_H: \text{coefficient of contact area } Z_H = \sqrt{\frac{2 \cos \beta}{\sin 2\alpha_{tw}}} = \sqrt{\frac{2 \cos 0}{\sin(2 \times 16.56^\circ)}} \approx 1.76$$

Z_ε : contact length coefficient, when $\varepsilon_b < 1$ use the following equation:

$$Z_\varepsilon = \sqrt{\frac{1}{\varepsilon_\alpha}} = \sqrt{\frac{1}{2.14}} = 0.79 \quad (6.36 \text{ c p.105[12]})$$

$$\text{where: } \varepsilon_\alpha = \frac{\sqrt{d_{a1}^2 - d_{b1}^2} + \sqrt{d_{a2}^2 - d_{b2}^2} - 2\alpha_w \sin \alpha_{tw}}{\frac{2\pi m \cos(\alpha_t)}{\cos(\beta)}} \quad (6.47 \text{ p105[12]})$$

$$= \frac{\sqrt{36^2 - 31.5^2} + \sqrt{19^2 - 16^2} - 2 \times 25 \times \sin(16.56^\circ)}{2\pi \times 1 \times \frac{\cos(16.56^\circ)}{\cos(0^\circ)}} = 2.14$$

$$\varepsilon_b = \frac{b_w \sin \beta}{m\pi} = 0 < 1 \quad (6.48 \text{ p105[12]})$$

$Z_M = 190 MPa^{\frac{1}{3}}$: material mechanical property coefficient, chosen based on table 6.5 p.96[1], given that the gears are made by steel

K_H : coefficient of load

$$K_H = K_{H\beta} K_{Hv} K_{H\alpha} = 1 \times 1.16 \times 1.038 \approx 1.22$$

where:

$K_{H\beta} = 1$ is coefficient of undistributed load on gear width

$K_{H\alpha} = 1.13$ is undistributed load coefficient

$$K_{Hv} = 1 + \frac{v_H b_w d_{w1}}{2T_1 K_{H\beta} K_{H\alpha}} = 1 + \frac{0.371 \times 7 \times 34}{2 \times 460 \times 1 \times 1.13} \approx 1.088$$

$$\text{with: } v_H = \delta_H g_0 v \sqrt{\frac{a_w}{u}} = 0.002 \times 73 \times 0.36 \times \sqrt{\frac{25}{0.5}} = 0.371$$

$\delta_H = 0.002$ is coefficient of contacting errors, chosen based on table 6.15 p.107[12]
given that gear with inclined teeth and $HB_2 \leq 350 HB$

$g_0 = 73$ is coefficient of teeth module error, chosen based on table 6.16 p.107[12]
given that the degree of accuracy is 9, module ≤ 3.55 .

Precisely calculating the allowable contacting stress:

$$[\sigma_H] = [\sigma_H] Z_R Z_V K_{xH} = 340 \times 1 \times 0.77 \times 1 = 260.6 \text{ MPa} \quad (6.1a \text{ p.93[12]})$$

where: $[\sigma_H]$ on the right side of the equation is calculated in part II.5 above,

$$[\sigma_H] = 340 \text{ MPa}$$

Checking the value of tangential stress:

$$\sigma_H \sqrt{K_{lv}} = 209.7 \times \sqrt{1} = 209.7 \text{ MPa} < [\sigma_{Hmax}] = 952 \text{ MPa}$$

→ This satisfies the strength condition

Determine the bending stress for driving gear:

$$\sigma_{F1} = \frac{2T_1 K_F Y_\varepsilon Y_\beta Y_{F1}}{b_w d_{w1} m} \quad (6. \text{ p.108[12]})$$

$$= 2 \times 460 \times 1.68 \times 0.47 \times 1 \times \frac{4.26}{(7 \times 34 \times 1)} = 12.5 MPa$$

where: $K_F = K_{F\beta} K_{F\alpha} K_{Fv} = 1.01 \times 1.37 \times 1.22 = 1.62$ is coefficient of load affected bending (the other parameters are chosen and calculated above)

$$Y_\varepsilon = \frac{1}{\varepsilon_\alpha} = 1/2.14 = 0.47 \text{ is the coefficient of meshing}$$

$$Y_\beta = 1 - \frac{\beta}{140} = 1 - 0/140 = 1 \text{ is the coefficient of teeth incline}$$

$Y_{F1} = 4.26$ is the coefficient of teeth type, chosen based on table 6.18 p.109[12] given the displacement coefficient $x = 0$ and the corresponding teeth of driving gear as calculated below:

$$z_{v1} = \frac{z_1}{\cos^3 \beta} = 34$$

$$K_{Fv} = 1 + \frac{v_F b_w d_{w1}}{2T_1 K_{F\beta} K_{F\alpha}} \quad (6.46 \text{ p.109[12]})$$

$$= 1 + \frac{4.07 \times 7 \times 34}{2 \times 460 \times 1.01 \times 1.37} \approx 1.22$$

where: the parameters are notated in the previous part

$$v_F = \delta_F g_o v \sqrt{\frac{a_w}{u}} = 1.11$$

Determine the bending stress for driven gear:

$$\sigma_{F2} = \frac{\sigma_{F1} Y_{F2}}{Y_{F1}} = 12.5 \times 3.72 / 4.26 = 10.9 MPa \quad (6.44 \text{ p.108[12]})$$

where: $Y_{F2} = 3.72$ is the coefficient of teeth type, chosen based on table 6.18 p.109[1], given the displacement coefficient $x = 0$ and the corresponding teeth of driven gear as calculated below

$$z_{v2} = \frac{z_2}{\cos^3 \beta} = 17$$

Calculating precisely the allowable bending stress for both gears:

$$[\sigma_{F1}] = [\sigma_{F1}] Y_R Y_S K_{xF} = 190.3 \times 1 \times 1.08 \times 1 \approx 205.5 MPa \quad (6.2a \text{ p.93[12]})$$

$$[\sigma_{F2}] = [\sigma_{F2}] Y_R Y_S K_{xF} = 174.9 \times 1 \times 1.08 \times 1 \approx 188.8 MPa \quad (6.2a \text{ p.93[12]})$$

where: $[\sigma_{F1}]$, $[\sigma_{F2}]$ on the right side of the equation is calculated,

$$[\sigma_{F1}] = 190.3 MPa; [\sigma_{F2}] = 174.9 MPa$$

$Y_R = 1$ is coefficient of surface roughness (use gears had been through grinding

process)

$Y_s = 1.08 - 0.0695 \ln(m) = 1.08 - 0.0695 \ln(1) = 1.08$ is coefficient of sensitivity to load

$K_{xF} = 1$ is coefficient of gear size affected to bending stress (when $d_a \leq 400$)

Checking the bending stress:

$$\sigma_{F1} K_{lv} = 235.2 \times 1 = 235.2 \text{ Mpa} < [\sigma_{F1max}] = 304 \text{ MPa} \quad (6.49 \text{ p.110[12]})$$

$$\sigma_{F2} K_{lv} = 232 \times 1 = 232 \text{ Mpa} < [\sigma_{F2max}] = 272 \text{ MPa} \quad (6.49 \text{ p.110[12]})$$

Both gears satisfy the bending stress condition

2.4 Design shaft

Although the robot does not carry heavy workload, C45 steel is chosen to ensure the stability.

Tangential force:

$$F_{t12} = \frac{2T}{d_{w2}} = \frac{2 \times 0.035}{0.017} = 4.12 \text{ (N)}$$

$$F_{t11} = \frac{2T}{R} = \frac{2 \times 0.035}{0.042} = 1.63 \text{ (N)}$$

Centrifugal force:

$$F_{r12} = \frac{F_{t12} \tan \alpha_{tw}}{\cos \beta} = \frac{4.12 \times \tan 20^\circ}{\cos 0^\circ} \approx 9.217 \text{ N}$$

$$F_{r11} = \frac{F_{t11} \tan \alpha_{tw}}{\cos \beta} = \frac{1.63 \times \tan 20^\circ}{\cos 0^\circ} \approx 3.64 \text{ N}$$

Moment on shaft:

Force equation on x axis:

$$\sum F_X = -F_{X11} + F_{X12} - F_{r12} - F_{t11} = 0$$

$$\rightarrow -F_{X11} + F_{X12} = F_{r12} + F_{t11} = 13.3 \text{ N}$$

Force equation on y axis:

$$\sum F_Y = F_{Y11} - F_{Y12} + F_{r11} - F_{t12} = 0$$

$$\rightarrow F_{Y11} - F_{Y12} = F_{t12} - F_{r11} = 0.48 \text{ N}$$

Moment equation in Ozx plane at A:

$$\sum M_{Ozx} = -F_{r12} \times 7 + F_{X12} \times 10 - F_{t11} \times 16.5 = 0$$

$$-9.217 \times 7 + F_{X12} \times 10 - 16.3 \times 16.5 = 0$$

$$F_{X12} \approx 33.35 \text{ N} \rightarrow F_{X11} = 20 \text{ N}$$

Chapter 2: Mechanical design

Moment equation in Ozy plane at A:

$$\sum M_{Ozy} = -F_{t12} \times 7 + F_{Y12} \times 10 - F_{r11} \times 16.5 = 0$$

$$-4.12 \times 7 + F_{Y12} \times 10 - 3.64 \times 16.5 = 0$$

$$F_{Y12} = 8.89 \text{ N} \rightarrow F_{Y11} = 9.37 \text{ N}$$

The other moments are calculated and presented in figure 2.5.

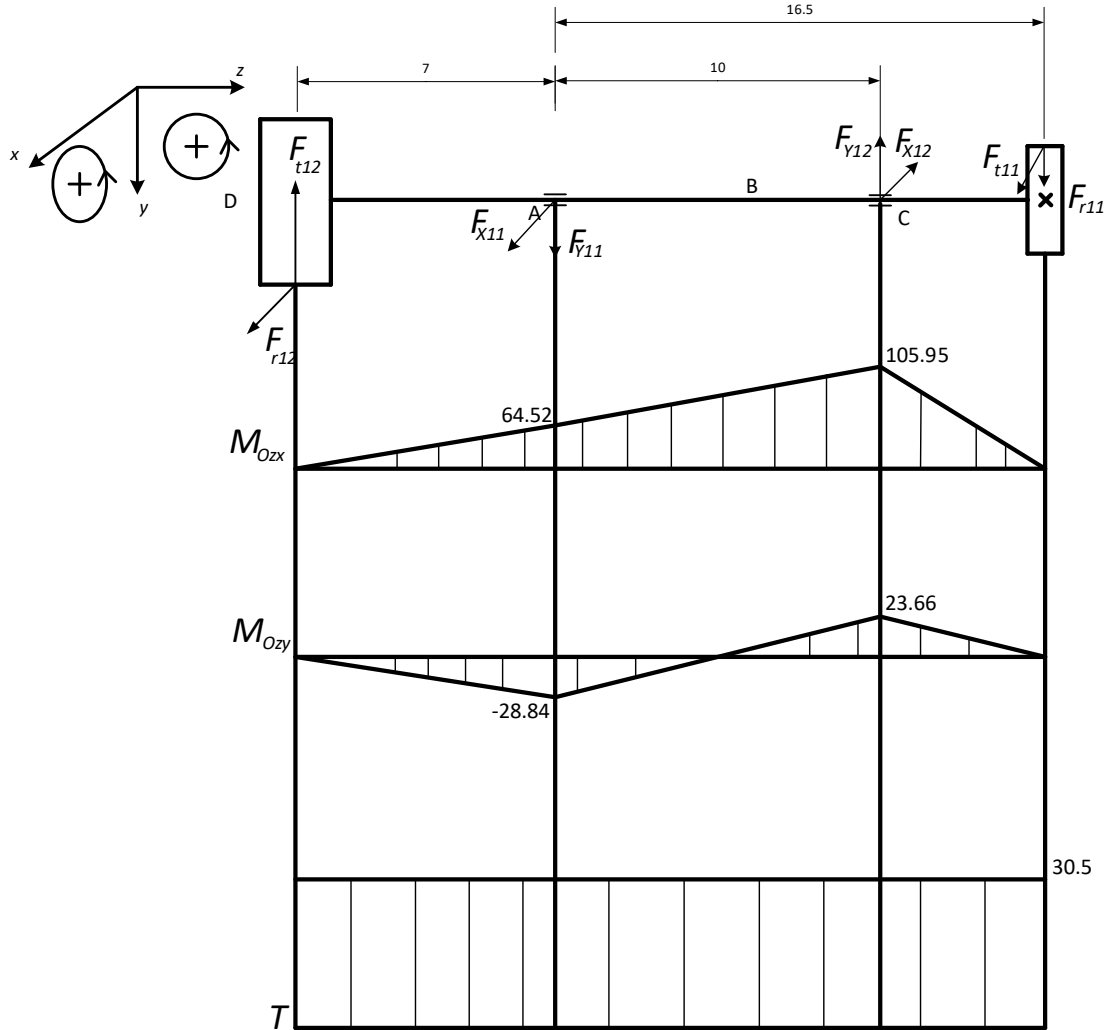


Figure 2.5 Shaft moment diagram

The most dangerous area is at C section, critical moment at C:

$$M_{td} = \sqrt{M_x^2 + M_y^2 + 0.75T^2} = \sqrt{105.95^2 + 23.66^2 + 0.75 \times 30.5^2} = 111.7 \text{ Nmm}$$

Diameter at section C:

$$d \geq \sqrt[3]{\frac{32M_{td}}{\pi[\sigma]}} = \sqrt[3]{\frac{32 \times 111.7}{\pi \times 82.5}} = 4.5 \text{ mm}$$

Choose shaft's diameter $\phi = 5 \text{ mm}$

2.5 Select bearing

Chapter 2: Mechanical design

Because there is no axial force applied on shaft, so choose row ball bearing

Longevity:

$$L = \frac{60nL_h}{10^6} = \frac{60 \times 400 \times 10000}{10^6} = 24(\text{million revolution})$$

with: $L_h = 10^4$ h is longevity of bearings in hours

n is the speed of shaft (rpm)

Force caused by bearing:

$$F_r = \sqrt{F_x^2 + F_y^2} = \sqrt{33.35^2 + 8.89^2} = 34.5 \text{ N}$$

Dynamic load sustainability:

$$C_d = Q \sqrt[m]{L}$$

with: Q is notational load (kN)

$$Q = (XVF_r + YF_a)k_tk_d$$

with: X is coefficient of centripetal load (chosen from table 11.4 p.215[12], based on practical coefficient e ; for shaft I and II, e is chosen based on the equation iF_a/C_o , with $i=1$ is the number of rows of the bearing; C_o is the ability to sustain stable load, chosen from table P2.12 p.264[12]; for shaft I, $e = 1.5 \tan(\alpha)$

Y is coefficient of axial load (chosen from table 11.4 p.215[12], the specifications for choosing are the same as X)

V is coefficient of which ring rotate ($V = 1$ when inner ring rotates)

k_t is coefficient of temperature $k_t = 1$ when temperature $\theta = 105^\circ$

k_d is coefficient of load specificity, $k_d = 1$ (chosen from table 11.3 p.215[1] for light impact load)

m is grade of the bending line, $m = 3$ for ball bearing

$$\rightarrow Q = (1 \times 1 \times 0.0345) \times 1 \times 1 = 0.0345 \text{ kN}$$

$$\rightarrow C_d = 0.0345 \times \sqrt[3]{24} = 0.0995 \text{ (kN)}$$

Hence, choose bearing having $C > C_d$

Choose 619/5 deep groove ball bearing

Bearing no	Boundary dimensions (mm)	Basic load ratings (kN)	Limiting speed (rpm)	Fatigue load limit (kN)

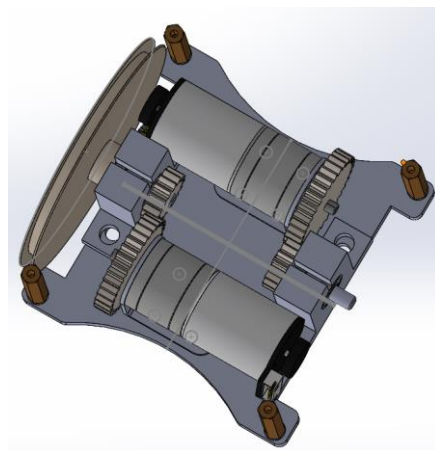
	d	D	B	r	C	C_o	n	P_u
619/5	5	13	4	0.2	0.884	0.335	70000	0.014

2.6 Evaluation

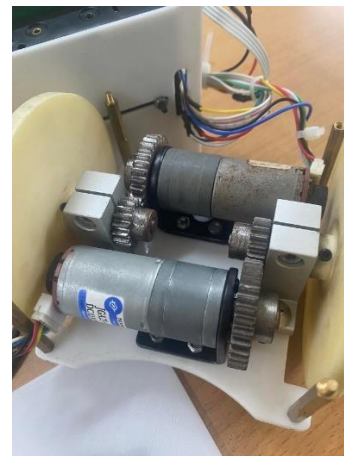
The gear transmission is fabricated after the design drawing is finished. Finally, the gear transmission completes the goal is to increase speed and force. However, the meshing accuracy is not extremely good. The reasons might be:

- The errors came from fabricated process.
- The center distance is not accurate because it depends on the accuracy of the 3D printed machine.

The robot has length side is larger than 100mm and reach at 106mm and the height side reach at 115mm. The other side meets the requirement, which is 100mm in length. Figure 2.6 reveals the mechanical final result.



a) 3D model



b) Real life model

Figure 2.6 Mechanical design result

CHAPTER 3: ELECTRICAL AND CONTROLLER DESIGN

The main purpose of this chapter is to design electrical hardware module for the robot and install them. The features of electrical module must satisfy the technical requirement, which are simple for proceed usage, suitable for small-size space optimization design. The achieved result of electrical module support to utilize the process of soccer robot controller design.

The main target of designing the controller for the robot is to control the speed of the motor with high stability and efficiency. To achieve the setting target, PID controller is one of the considerable controllers, because of the simplicity, accuracy and stability, and the most important point is to execute. The PID controller design must optimize the control loop for response time, accuracy and stability under all process conditions.

General of Electrical design

Electrical principal diagram includes 3 main modules:

- Power module: Supply energy for motor and microcontroller
- Microcontroller module: execute the source code
- Driver module: Receive signals from microcontroller and control motors

Figure 3.1 illustrates the electrical principal diagram of the robot.

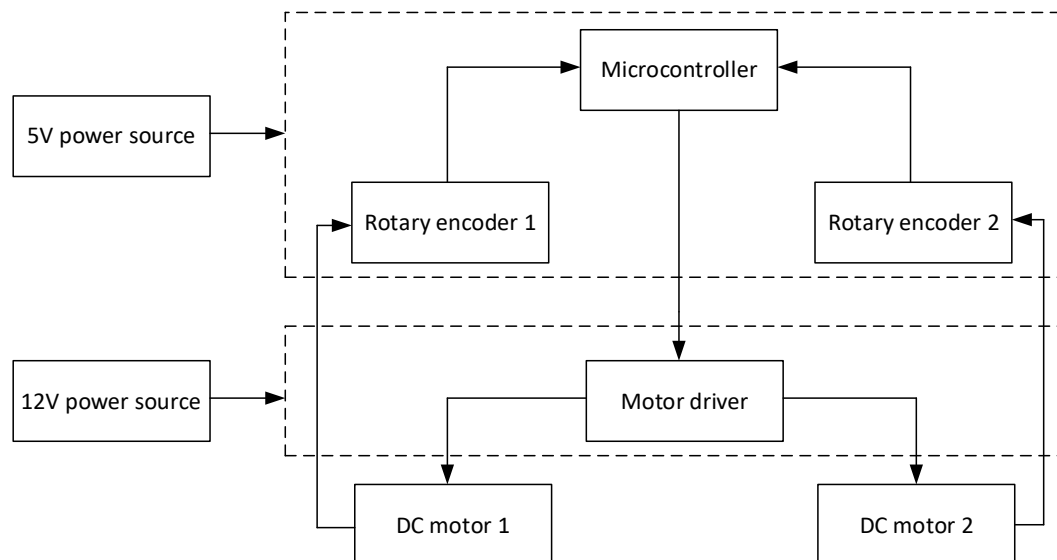


Figure 3.1 Electrical principal diagram

3.1 Driver and Motor module

From the motor characteristics mentioned in chapter 3, selected driver should

Chapter 3: Electrical and Controller design

satisfy several technical requirements:

- Power supply for motor is 12V.
- Driver should conduct 1A.
- Driver frequency control should be suitable for PWM frequency of microcontroller.
- Driver size is as small as possible.

Driver products line is being used popularly to control DC motors such as L298N or TB6612FNG. Based on the size and performance requirements, choose TB6612FNG. Figure 3.2 is motor control block, which illustrates the connection between driver and 2 motor GA25 and figure 3.3 illustrates driver pin definitions.

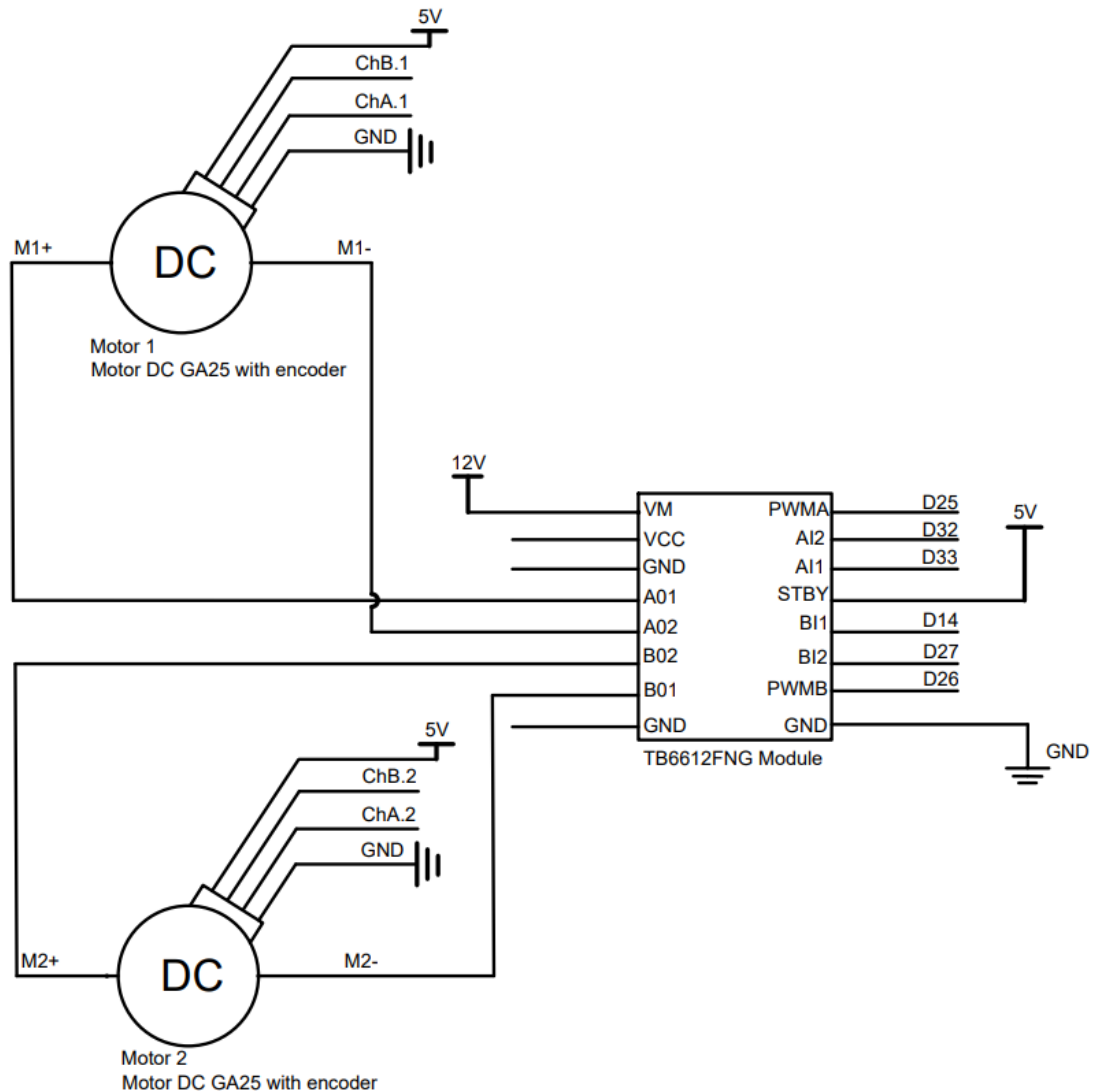


Figure 3.2 Motor control block

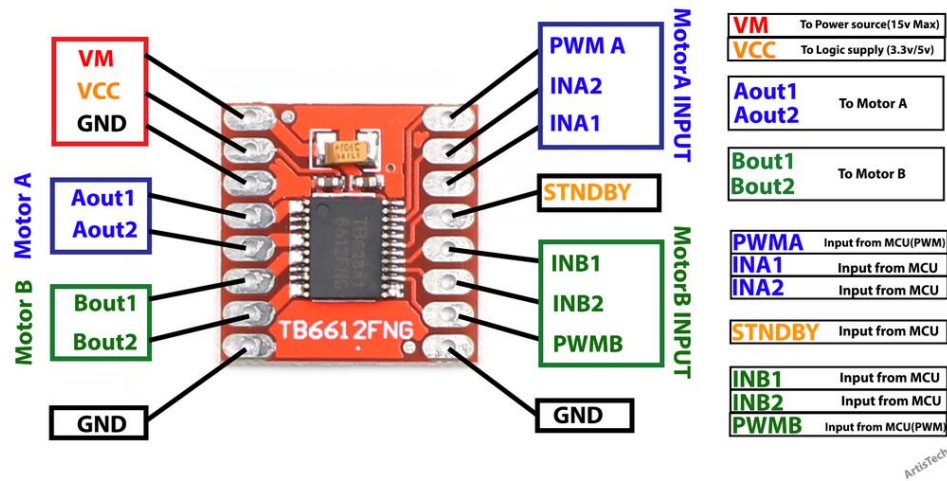


Figure 3.3 TB6612FNG driver [13]

Table 3.1 TB6612FNG driver datasheet

Driver \ Feature	Power supply voltage (V)	Output current (A)	Maximum control frequency (kHz)	Number of motors can be controlled simultaneously
TB6612FNG	4.5-15	1.2-3.2	100	2

3.2 Microcontroller module

Technical requirements:

- Microcontroller should have the baud rate high enough to meet the demand of real time problems.
- The size of microcontroller should as small as possible.
- High performance is required to execute a huge calculation in source code.
- Support for TCP/Wi-Fi protocol.
- Utilizing MicroPython library.

The Esp32 chip comes with 48 multiple functions. Not all pins are exposed in all Esp32 development board, and some pin cannot be used. To achieve the requirement, Esp32 Devkit V1 consists of all the needs of the project. In the figure 3.4, the figure describes Esp32 Devkit V1 pin definitions.

The ESP32 peripherals include in the table 3.2 below. The ADC (analog to digital converter) and DAC (digital to analog converter) features are assigned to specific static pins. However, you can decide which pins are UART, I2C, SPI, PWM, etc to use this function, need to assign them in the code. This is possible due to the ESP32 chip's multiplexing feature. Figure 3.5 describes the control block of the Esp32 Devkit V1.

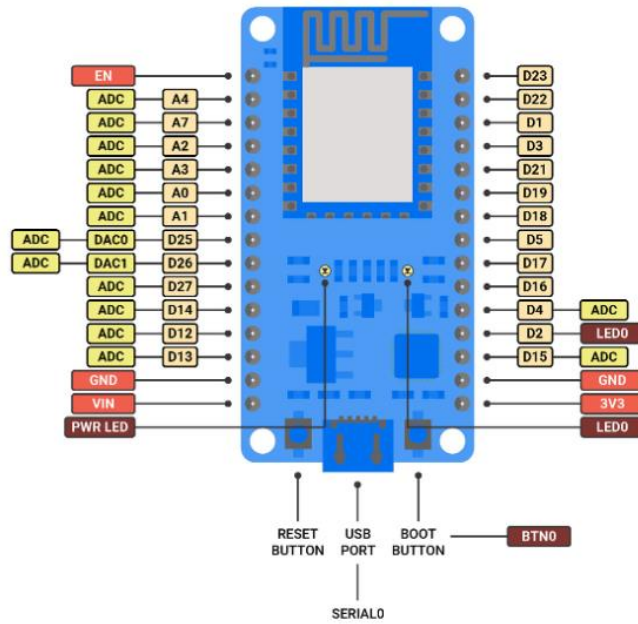


Figure 3.4 ESP32 DEVKIT V1 pinout diagram [14]

Table 3.2 Esp32 datasheet [22]

Parameter	Value
WiFi	2,4GHz; 802.11 b/g/n
Maximum baud rate	115200
Required voltage	5V
Energy consumption	<1,0mW
Flash memory	4MB
Protocols	TCP/IP
Maximum peripherals	5
Digital I/O pins (DIO)	25
Analog Input Pins (ADC)	6

Analog Output Pins (DAC)	2
UARTs	3
SPIs	2
I2Cs	3

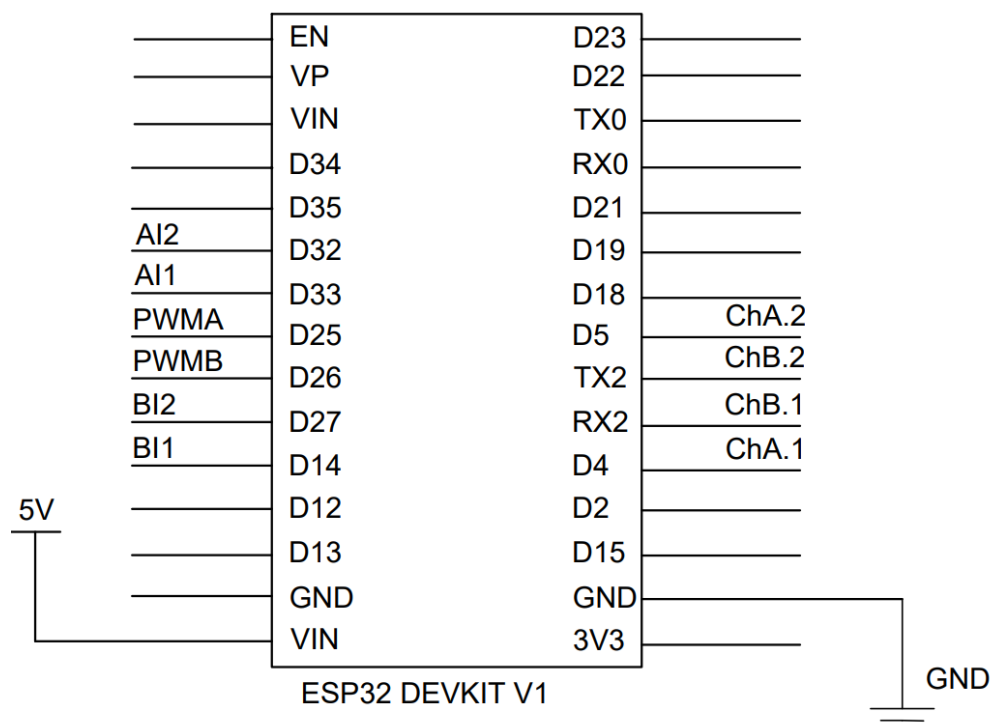


Figure 3.5 Controller block

3.3 Buck converter

The microcontroller need only 5V and the driver need only 12V to operate, hence the presence of buck converter is unavoidable.

Requirements:

- The buck converter can convert power source to 12V source and 5V source.
- The buck converter size should be compact.
- The cost is affordable.

Conclusion: Choose LM2596 buck module

Table 3.3 LM2596 buck module parameter

Parameter	Value
Input voltage	4V-35V
Output voltage	1.23V-30V
Output current	3A (max)
Efficiency	92% (max)
Frequency	150kHz
Working under temperature	$-40^{\circ}C - 85^{\circ}C$
Size	4.3 x 2.1 x 1.7 cm



Figure 3.6 LM2596 buck converter [15]

3.4 Power module

The power module should be compact and can handle to supply power to all devices in operating process.

Table 3.5 Total power of the electrical system

Devices	Quantity		Current	Voltage	Power
GA25 DC motor	2		2.5A	24V	60W
Total power					60W

Total power supply is 60W.

The safety coefficient is $k = 1.5$, the minimum power of power module can be calculated:

$$Q_{min} = 1.5 \times W \times T = 1.5 \times 60 \times 8 = 720 (Wh)$$

Required minimum capacity for the power module:

$$Ah_{min} = \frac{Q_{min}}{V} = \frac{720}{24} = 30 (Ah)$$

Combine the power calculation results with the 8 hours per day require, the Lithium Li-on battery 24V - 20Ah with maximum discharging is 3A is selected. After pins run out of power, charging is required. Pin case and pins are demonstrated in figures 3.7 and 3.8 respectively. Figure 3.9 illustrates the power module wiring.

Table 3.6 Lithium Li-on battery parameters

Parameters	Value
Voltage	24 V
Maximum discharging	3 A
Capacity	20 Ah
Dimension	18 x 11 x 7 cm
Mass	0.2 Kg



Figure 3.7 Lithium Li-on pin case



Figure 3.8 Lithium Li-on pin

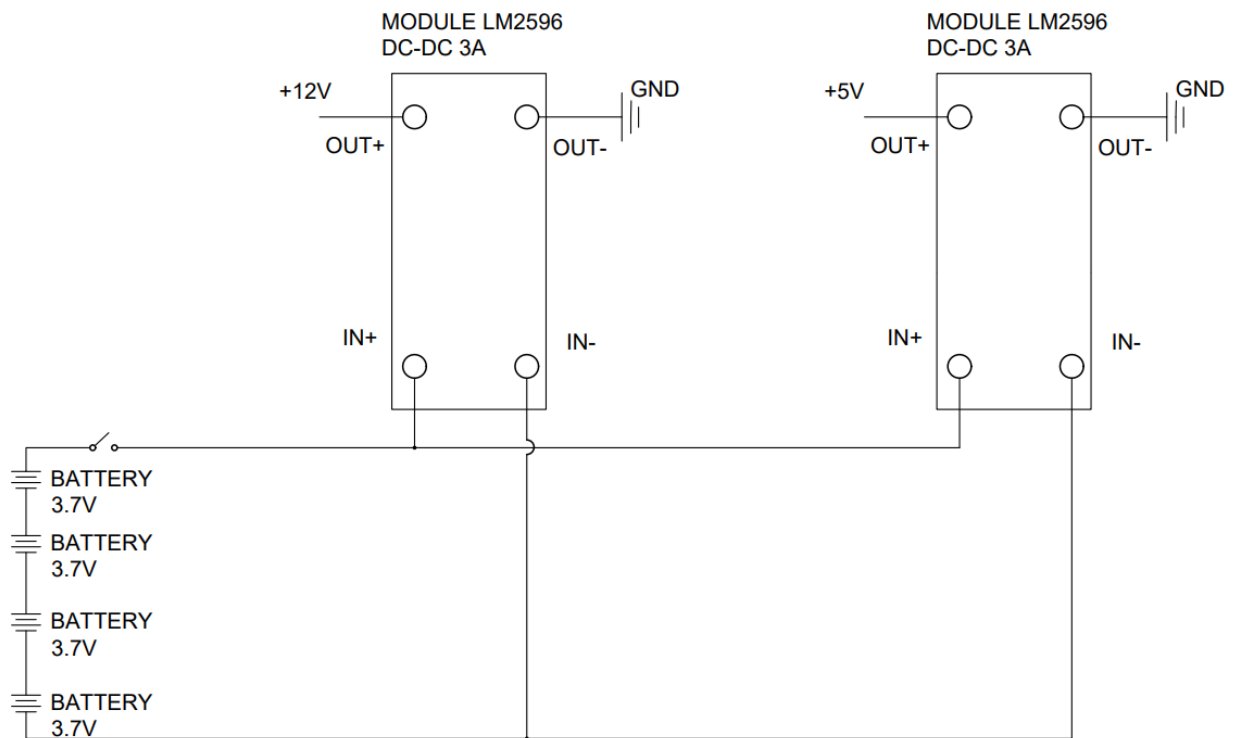


Figure 3.9 Power module

3.5 Printed Circuit Board (PCB)

A printed circuit board (PCB) is an electronic assembly that uses copper conductors to create electrical connections between components. Printed circuit boards provide mechanical support for electronic components so that a device can be mounted in an enclosure. A printed circuit board design must include a specific set of steps that aligns with the manufacturing process, integrated circuit packaging, and the structure of the bare circuit board.

For the need of small size robot, the PCB is designed to have a square size of 100 mm^2 . Beside that, for the technical flow and easily to process, PCB has to be designed to simply place the electrical components and easy to coding.

The PCB is fabricated by the Chinese after the design process is finished from easily design from the electrical drawing stuff. The manufacture processed the design PCB board in technical drawing electrical board's software for example: Altium, Ansys,... The final PCB board has shown in figure 3.10.

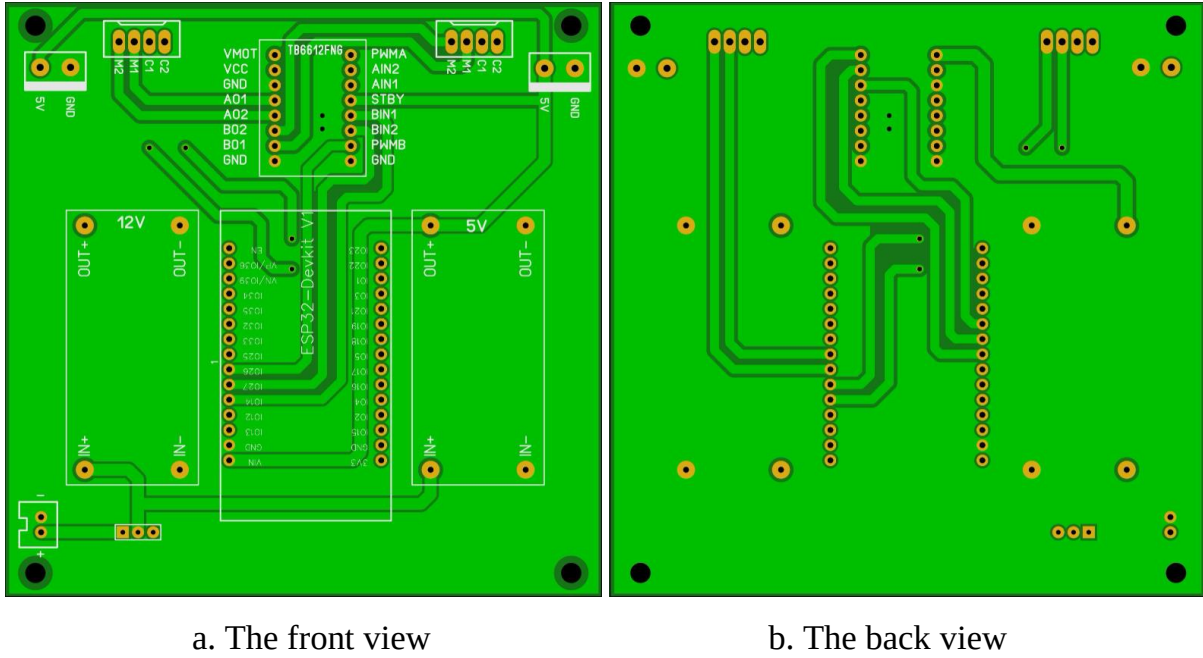


Figure 3.10 Printed circuit board (PCB)

3.6 Mathematic model

Because the robot run on an even ground, constant load, no external force during operation process, so only kinematic analysis should be considered. Figure 3.11 reveals the kinematic model of the robot.

The position and orientation of the robot can be displayed by 3 main parameters (x, y, φ) . In which, (x, y) is coordinate of the robot and φ is the angle formed by a vector which is fixed to the robot and a fixed vector in the fixed coordinate system.

Assume v and ω are velocity and angular velocity respectively at centroid C. Kinematic equation of the system:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\varphi} \end{bmatrix} = \begin{bmatrix} \cos \varphi & 0 \\ \sin \varphi & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (3.1)$$

Angular velocity of the wheel:

$$\begin{bmatrix} \omega_r \\ \omega_l \end{bmatrix} = \begin{bmatrix} \frac{1}{r} & \frac{b}{r} \\ \frac{1}{r} & -\frac{b}{r} \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (3.2)$$

In which,

ω_r : right wheel's angular velocity

ω_l : left wheel's angular velocity

r : wheel radius

b : distance between wheels

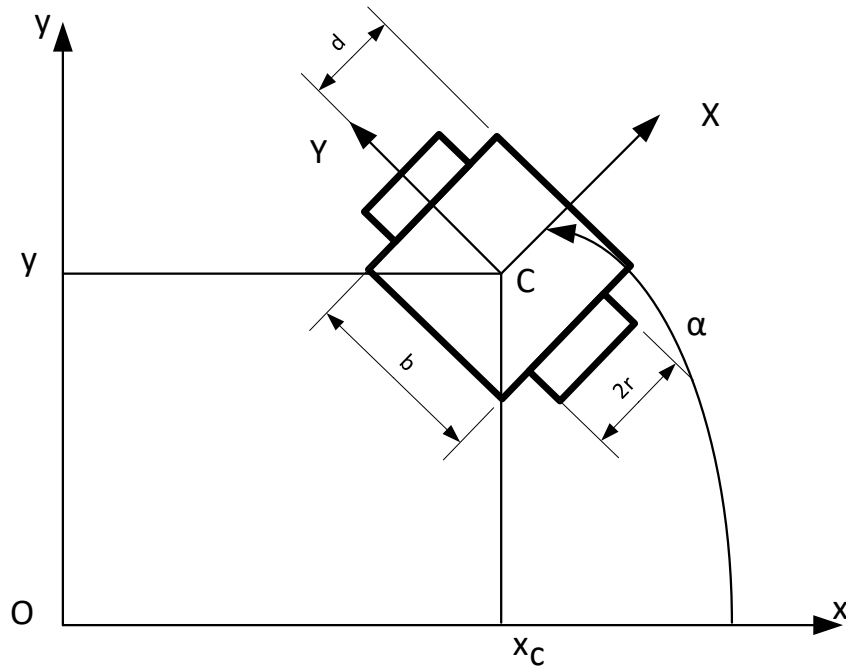


Figure 3.11 Kinematic model

Tracking point coordinate:

$$\begin{cases} x_r = x_c + d \cos \phi \\ y_r = y_c + ds \end{cases}$$

In which,

d : distance from centroid to tracking point

To control the robot to follow the virtual line path $R(x_r, y_r, \phi_r)$, the centroid must be controlled to follow the desire line path and the direction of the robot is tangent to the line. Figure 3.12 reveals the error model of the robot.

Robot followed to the line satisfies:

$$\begin{cases} x_r = v_r \cos \phi_r \\ y_r = v_r \sin \phi_r \\ \phi_r = \omega_r \end{cases} \quad (3.3)$$

To meet the demand, 3 characteristic error parameters are required to demonstrate the error of distance from centroid to R line, $e = [e_1 \ e_2 \ e_3]^T$

The errors are determined by geometry relation:

$$\begin{bmatrix} e_1 \\ e_2 \\ e_3 \end{bmatrix} = \begin{bmatrix} \cos \varphi & \sin \varphi & 0 \\ -\sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_r - x \\ y_r - y \\ \varphi_r - \varphi \end{bmatrix} \quad (3.4)$$

Differentiate the equation 3.3,

$$\begin{bmatrix} \dot{e}_1 \\ \dot{e}_2 \\ \dot{e}_3 \end{bmatrix} = \begin{bmatrix} v_r \cos e_3 \\ v_r \sin e_3 \\ \omega_r \end{bmatrix} + \begin{bmatrix} -1 & e_2 \\ 0 & -e_1 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (3.5)$$

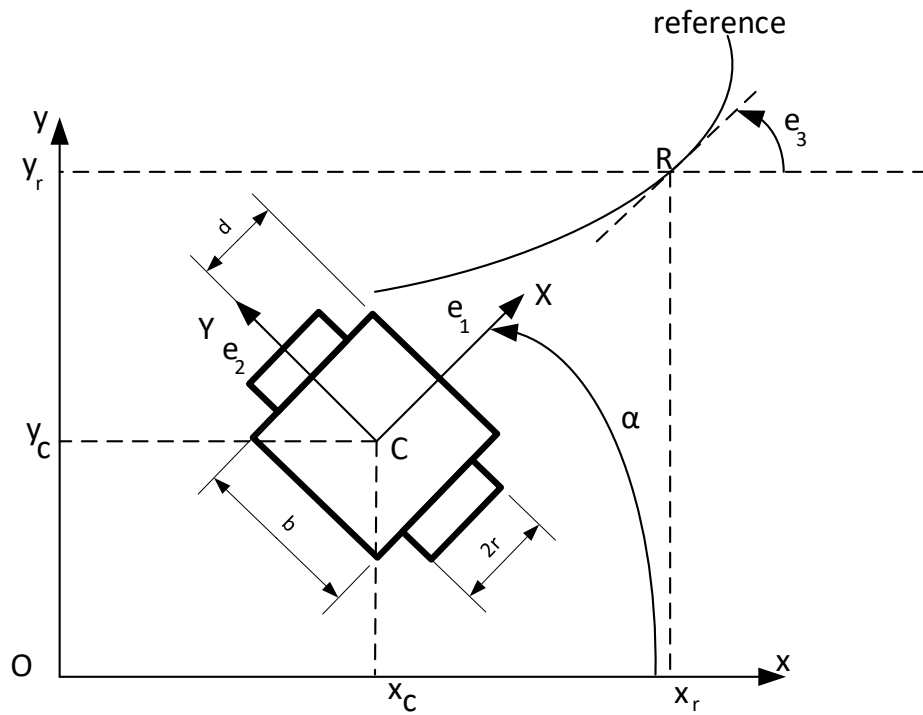


Figure 3.12 Error model

3.7 PID controller design

Motor is controlled by closed-loop feedback PID controller, in which the input parameter is a desired velocity v_{ref} and receive a real-life velocity v , which is feed back by encoder pulses. The controller block is described as the figure

Assume that motors and driver as a block, the relationship between the input signal and the output signal are calculated by supplying PWM pulses and record the velocity values read from encoder pins. Table 3.7, 3.8 describe the recorded result of right motor and left motor respectively. Moreover, figures 3.13 and 3.14 demonstrate the trend of motor velocity correspond to each PWM value.

Table 3.7 Revolution per minute correspond to %PWM (Right motor)

Number	PWM (%)	Speed (rpm)
1	10	40.3
2	15	65.5
3	20	91.5
4	25	115.5
5	30	147.6
6	35	176.5
7	40	194.1
8	45	223.2
9	50	255.1
10	55	282.4
11	60	308
12	65	332.1
13	70	359.4
14	75	388.2
15	80	420.3
16	85	447.6
17	90	470
18	95	500.1
19	100	521.4

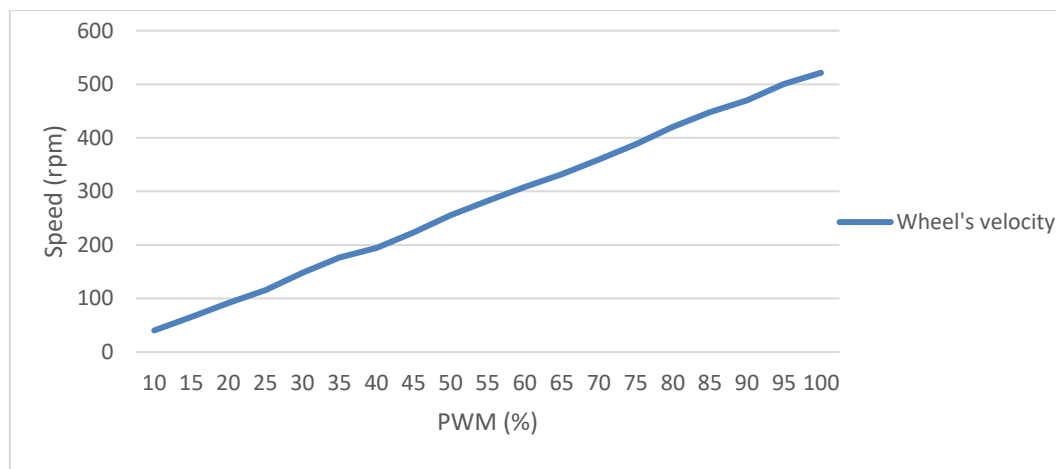


Figure 3.13 Revolution per minute correspond to %PWM (Right motor)

Table 3.8 Revolution per minute correspond to %PWM (Left motor)

Number	PWM (%)	Speed (rpm)
1	10	38.5
2	15	59.4
3	20	94.7
4	25	104.3
5	30	133.2
6	35	160.5
7	40	187.7
8	45	208.6
9	50	243.9
10	55	267.9
11	60	296.8
12	65	322.5
13	70	351.3
14	75	375.4
15	80	401.1
16	85	432
17	90	454
18	95	481.3
19	100	497.4

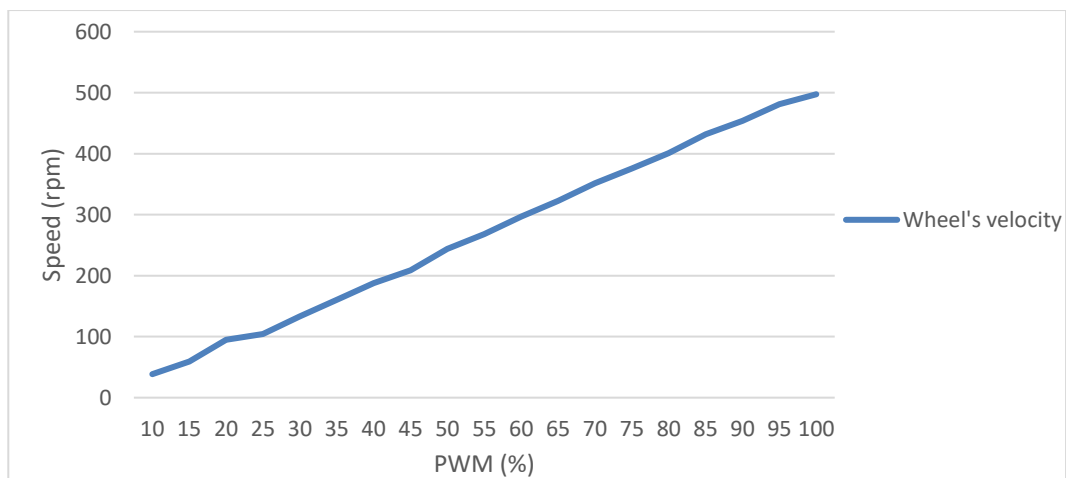


Figure 3.14 Revolution per minute correspond to %PWM (Left motor)

From the results, taking advantages from tools, which are System Identification and

Chapter 3: Electrical and Controller design

PID Tuner from Matlab to determine the motor transfer function and move to finding PID parameters step.

Pulse values need to be supplied in 1 cycle is described in the formula below:

$$u = K_p \cdot e + K_I \cdot \int e dt + K_D \cdot \frac{de}{dt}$$

In which:

- K_p, K_I, K_D are PID controller parameters
- $e = v_{ref} - v$ is the error between the real-life velocity and desired velocity
- $dt \approx 0.1s$ is sampling time

With 2 set of different PID parameters, 2 motors have a number of different responses in settling time and the overshoot. Hence, although the moment when 2 motors have the same values of velocity, the robot real-life trajectory is different from desired one. Moreover, the robot has shaking phenomenon from the start position. Hence, the controller needs to meet the demand in final requirement to ensure the balance of 2 motors from the start point:

- Overshoot: 7%
- Settling time is the same

All sets of PID parameters are described in table 3.9

Table 3.9 Set of PID parameters

	K_p	K_I	K_D
Left motor	2.5	10	0
Right motor	2.5	10	0

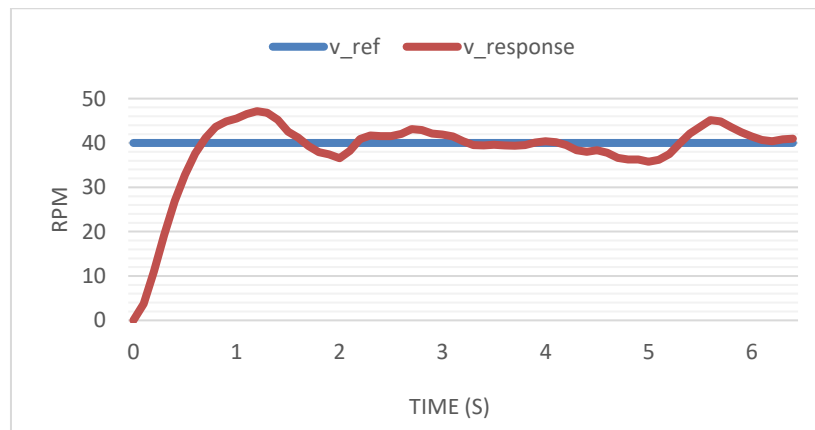


Figure 3.15 Left motor response PID

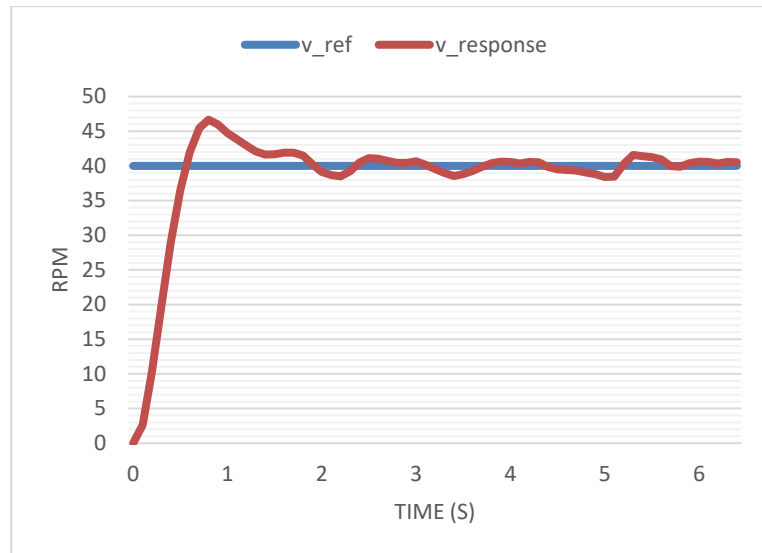


Figure 3.16 Right motor response PID

After all PID parameters are found, using them to test motors in reality, desired velocity of each motor is $v_{ref} = 40rpm$ with the sampling time is $t_s = dt \approx 100ms$. Figures 3.13 and 3.14 describe the real-life responses.

3.8 Evaluation

3.8.1 Electrical hardware

After the PCB is delivered, all the electronics are installed, and the figure 3.17 is the final result. The PCB is then supplied power by the 14.8 power source and after checking by VOM, testing had no weird phenomenon occurred. Figure 3.17 is the final electric hardware result.

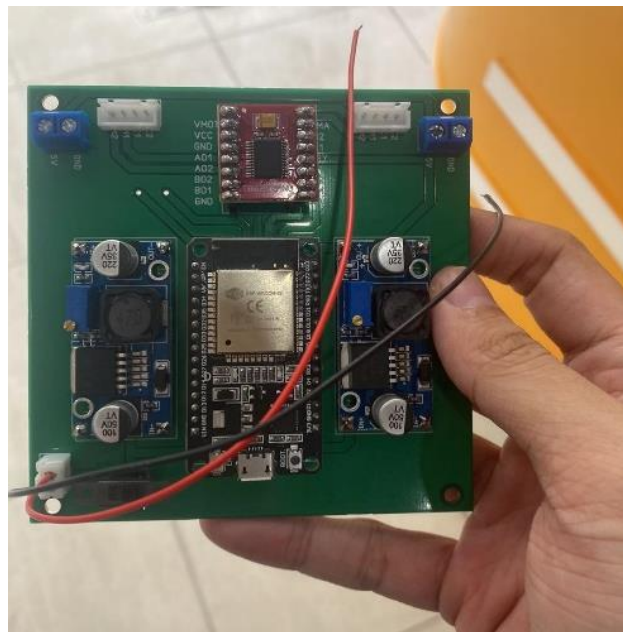


Figure 3.17 Final printed circuit board (PCB)

3.8.2 PID controller

After finishing the installation of hardware, the controller is tested, followed by 1 case: moving forward. In case moving forward, the robot is controlled to move forward with 10cm of distance. The measuring process can be summarized as follow:

- Firstly, manually rotate the wheels to push the robot to move 10 centimeters. The purpose of this step is to calculate exactly how many pulses per centimeter. In this step, a counting pulse function is required. Figure 3.18 illustrates the process of step 1. Table 3.10 records the results of each attempt

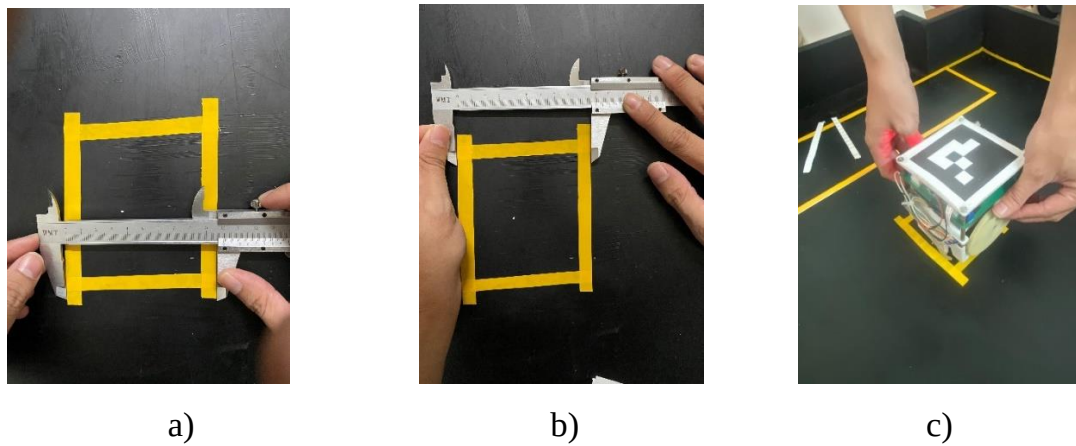


Figure 3.18 Counting of rising pulses process

a), b) Recheck the distance **c)** Push the wheels to move 10 centimeters

Table 3.10 Counting rising pulses record

Attempt	Number of rising pulses
1	82
2	78
3	79
4	83
5	80
Average	80

- Secondly, input the average number of pulses from table 3.10 to recheck whether the robot move forward a distance of 10 centimeters. Caliper ruler is used to check the length of distance the robot has moved, which is demonstrated in figure 3.19. Table 3.11 records the result of step 2.

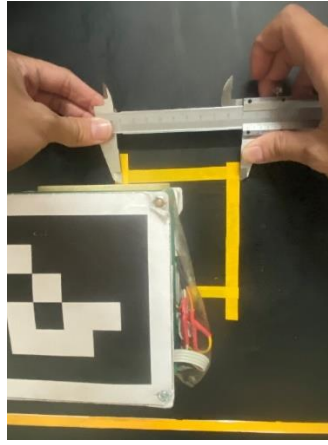


Figure 3.19 Recheck the distance

Table 3.11 Recheck the distance result

Case	Moving forward d (cm)
1	9.42
2	9.36
3	9.01
4	9.53
5	9.35
6	9.26
7	9.12
8	9.55
9	9.28
10	9.67
Average	$\bar{d} = 9.36$

From data in table 3.10, the standard deviation can be determined

$$\sigma_d = \sqrt{\frac{1}{9} \sum_{i=1}^{10} (d_i - \bar{d})^2} \approx 0.21(cm)$$

Then the standard error can be calculated:

$$\sigma_{\bar{d}} = \frac{\sigma_d}{\sqrt{10}} \approx 0.07(cm)$$

Based on 3σ standard with 97% of reliability, the errors are quite large.

3.9 Conclusion

The electrical module hardware operates with high stability and no risk of fire explosion and interruption, which leads to easily design the robot controller. However,

Chapter 3: Electrical and Controller design

the robot controller has several limitations such as the errors between the desired speed and real-life speed are quite large.

The sources of errors might come from:

- The surface of the field is not ideal.
- The errors from gear transmission.
- The PWM signal sent to motor might be noised.

From these limitations of robot controller, computer vision module is an important part of the system to enhance the controller performance.

CHAPTER 4: COMPUTER VISION

As mentioned from previous chapter, the computer vision is an important feedback module for the controller to control the robot to move to a desired positions with high accuracy. The goals of computer vision module are:

- Ball detection
- Robot recognition
- Convert from frame coordinate to real-life coordinate

4.1 Selecting camera

Due to the project's scope, the suitable web camera should have some features and characteristics as below:

- High fps and resolution while streaming
- Able to auto focus
- Support for window operation system

The specifications are given as:

- Depth of field: 5mm
- Field of view: area of the soccer field, which is 800x1000 mm (this includes the goal area)
- Smallest feature: a square of size 5mm
- At least 4 pixels in one smallest feature

The camera resolution is estimated as:

- Number of pixels on vertical axis = $4 \times \frac{800}{5} = 640$ pixles
- Number of pixels on horizontal axis = $4 \times \frac{1000}{5} = 1000$ pixles



Figure 4.1 C922 Pro HD stream webcam

Chapter 4: Computer vision

Then the frame rate is calculated as:

$$\text{Frame rate} = \left(\frac{2 \times \text{Smallest features}}{\text{Speed}} \right)^{-1} = \left(\frac{2 \times 5 \text{ mm}}{0.1 \text{ m/s}} \right)^{-1} = 10 \text{ fps}$$

Hence, C922 Pro HD stream webcam, which is describes in figure 4.1, would be one of considerable choices. The camera characteristics are mentioned in table 4.1.

Table 4.1 Technical Specification C922 Pro HD stream webcam

Features	Quality
FPS	Full HD 1080p/30fps HD 720p/60fps
Camera angle	78°
Focus type	Auto focus
Connection	USB-A 2.0 connected to the USB Virtual COM Port (VCP) – 12 Mbits/s
Wire length	1.5 m
Brightness sensor	Available
Base	Available

4.2 Camera installation

The camera position should be at suitable height in order to receive the image which covers all the soccer field dimensions. Besides, it must be at the center of the field due to the detection algorithm condition. Figure 4.2 describes the camera installation model.

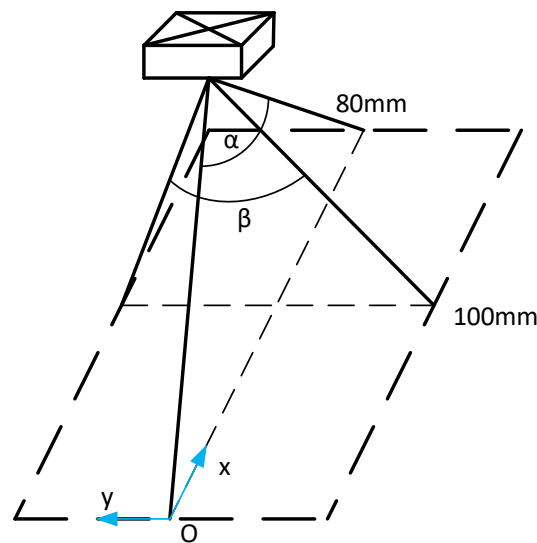


Figure 4.2 Camera installation model

According to Ox:

$$h = \frac{50}{\tan\left(\frac{78}{2}\right)} \approx 617mm$$

According to Oy:

$$h = \frac{40}{\tan\left(\frac{78}{2}\right)} \approx 493mm$$

Hence, choose $h = 800mm$, so the camera can cover all the soccer field.

The camera is installed on a shaped aluminum mechanism.

4.3 Image processing

4.3.1 Image processing procedure

The image processing procedure is demonstrated in the figure 4.3.

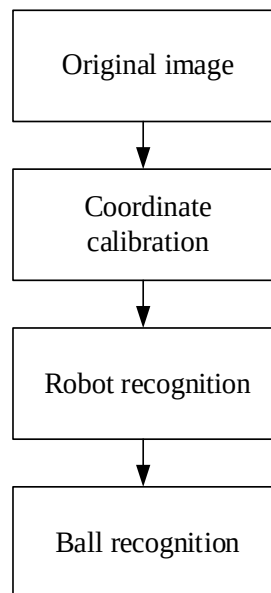


Figure 4.3 Image processing workflow

4.3.2 Coordinate calibration

The purpose of coordinate calibration is to reduce the effort to control the coordinate of objects or obstacles. The original is put at the middle of the field position, and the coordinates are converted from pixels to centimeters. Figure 4.4 describes the expected output for the coordinate calibration. The coordinate calibration process can be described in several steps:

- First, the field edges are recognized, and 4 corners coordinates are determined with pixels unit. The calculation of each edge is then taken.
- Next, the pixel per centimeter ratio (ppc) is calculated by dividing the edge

length in pixel to the edge length in centimeter.

- After that, translating the position of objects corresponding to the new origin, then dividing all the coordinates in pixels to ppc ratio.

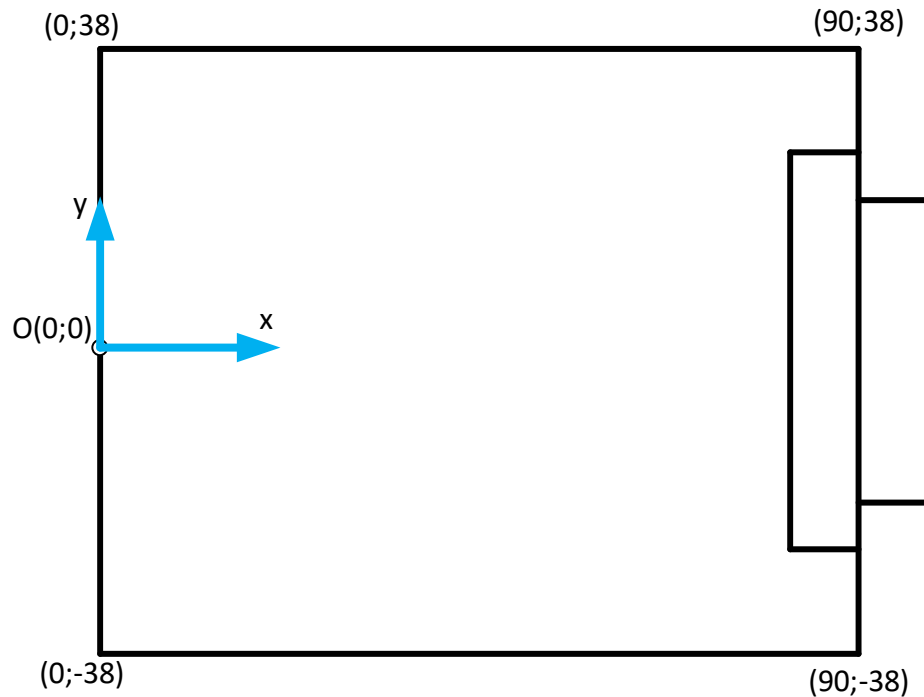


Figure 4.4 *Expected Coordinates after Calibration*

4.4 Robot recognition

The purpose is to determine the position and orientation of the robot. Based on robot shape and finding correspondences between points in the real environment and their 2d image projection process, using Aruco marker is one of the noticeable choices. Furthermore, recognizing multiple robots and analyzing their coordinate parameters on the field problem can be handled.



Figure 4.5 *ArUco marker used in this project*

An Aruco marker is a synthetic square marker composed by a wide black border

and an inner binary matrix which determines its identifier (id). The black border facilitates its fast detection in the image and the binary codification allows its identification and the application of error detection and correction techniques. The marker size determines the size of the internal matrix. Figure 4.5 shows the Aruco marker using in this project, and figure 4.6 illustrates the marker detection workflow.

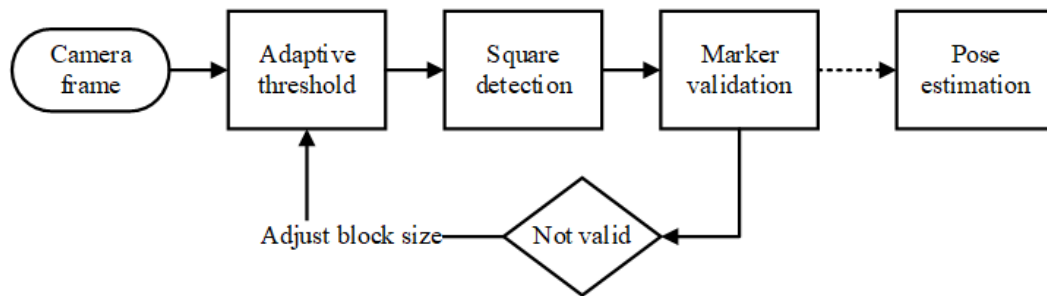


Figure 4.6 Marker detection workflow

By choosing Aruco marker as a recognition method, the advantages from the Aruco library can be taken. Some important functions used in computer vision module are `aruco.Dictionary_get` function, which used to create Aruco dictionary with desired bits size, or `aruco.detectMarkers` function, which used to recognize the Aruco ID on the soccer field. Moreover, after all necessary parameters are detected, they are stored in a text file.

4.5 Ball recognition

To maximize the reliability of the ball recognition, there is 2 main steps should be taken: Recognize the color and recognize the circle shape.

The chosen ball should satisfy technical requirement in the project's scope, which are standard size and weight. Moreover, to facilitate the color detection problem, its color should be different from the background. The ball used for the project is Inesis golf ball, which is described in figure 4.7, with 43mm diameter and yellow color.



Figure 4.7 Inesis golf ball

4.5.1 Color detection

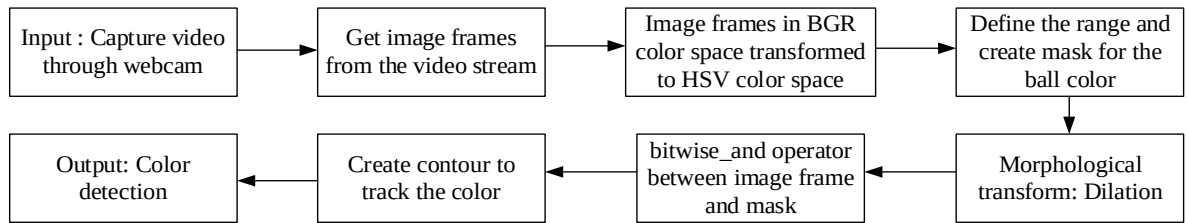


Figure 4.8 Color detection workflow

The color detection should be taken because after this step, only the circle shape of the ball would be captured, which is easier for circle detection process to step in. There are several useful libraries, which can be applied. They are Numpy library for numerical processing and cv2 for the OpenCV bindings. Figure 4.8 shows the workflow of color detection and figure 4.9 shows the result.



Figure 4.9 Color detection result

4.5.2 Circle detection

The next step is to detect the circle shape from bitwise_and operator between image frame and mask, then the coordinates are calculated. The Hough circle function, which can return the coordinate of the circle shape, from cv2 library would be one of the best choices. The main advantage of this function is that it is insensitive to occlusion. Figure 4.10 shows the circle detection workflow, and figure 4.11 shows the final result of recognizing the ball.

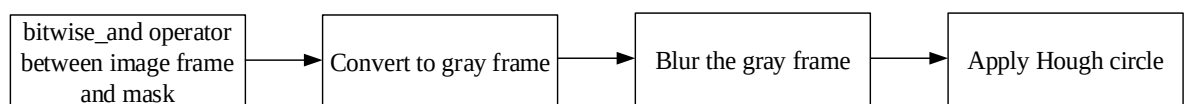


Figure 4.10 Circle detection workflow

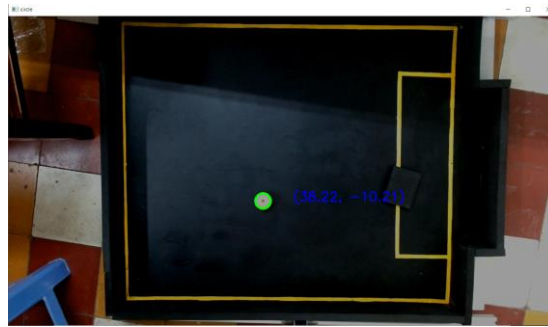


Figure 4.11 Final result of recognizing the ball

4.6 Flowchart

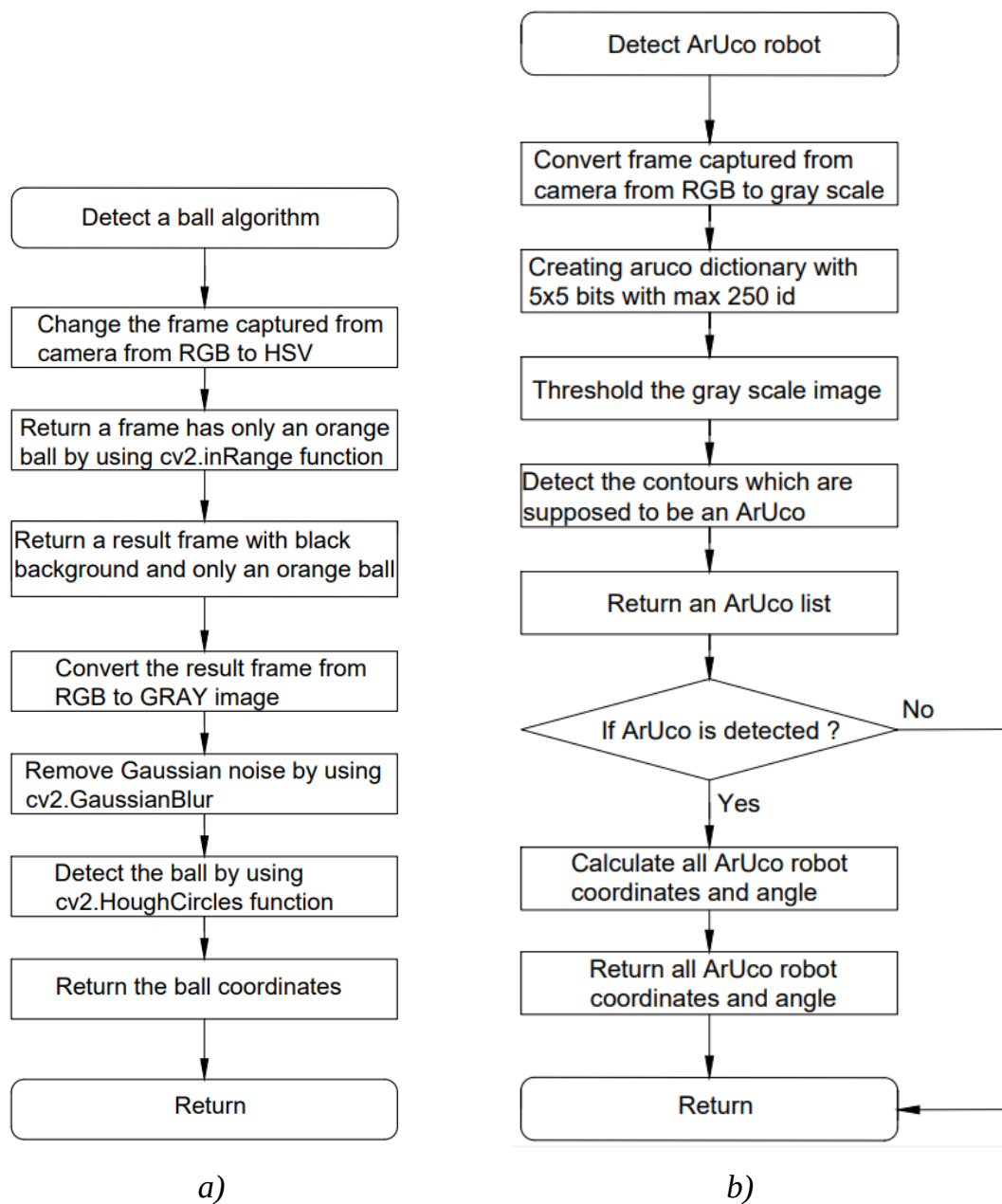


Figure 4.12 Flow chart of sub program

a) Ball detection b) Robot detection

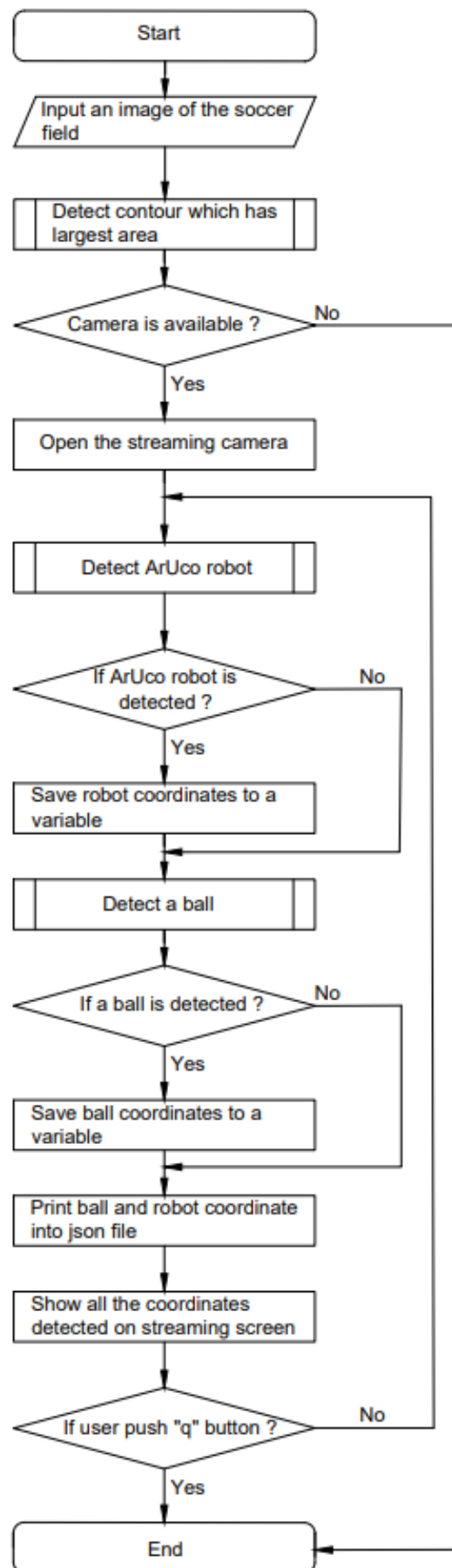


Figure 4.13 Final computer vision flowchart

Chapter 4: Computer vision

At the beginning, a bird-eye view of soccer field should be captured and is used as an input for the algorithm. Moreover, after the ball and robot are recognized, the coordinates of the ball will be displayed on screen and printed into a text file. Figure 4.12 illustrates 2 subprogram flowcharts, which are used to detect ball and robot position, and figure 4.13 describes the main program.

4.7 Evaluation

After all the objects are enabled to be detected, the accuracy should be checked. Based on ball and soccer field dimension, the first checking method is to put the ball at 4 different soccer field corners. The purpose is to check whether the position of the objects received by software are extremely accurate. Figure 4.14 illustrates the checking positions, and the errors of each case are recorded in table 4.2.

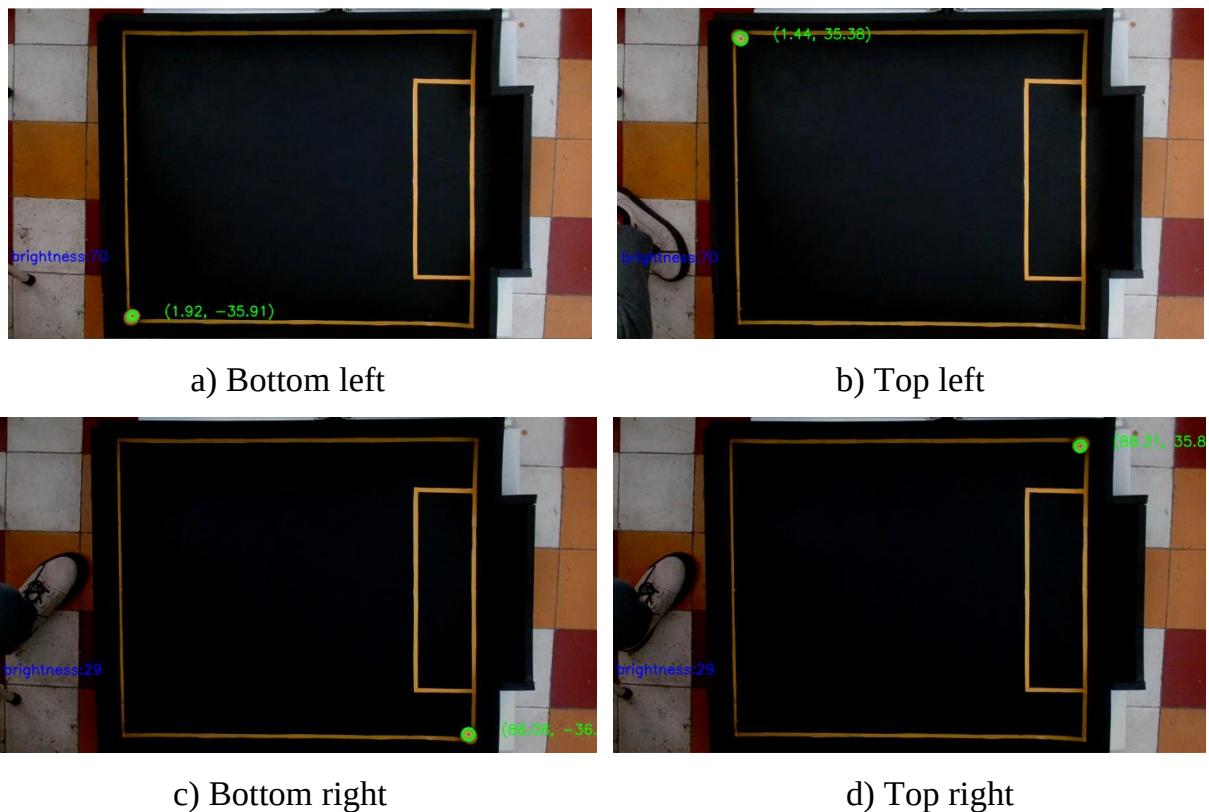


Figure 4.14 Ball positions

Table 4.2 Ball recognition error result (first method)

	Ox direction (cm)	Oy direction (cm)
Bottom left	0.23	0.06
Top left	0.71	0.47
Bottom right	0.23	0.25
Top right	0.46	0.05

Chapter 4: Computer vision

Similarly, the positions of the robot, which have the highest errors, are also captured and described by figure 4.15. Moreover, the accuracy of the robot can be evaluated, which is revealed in table 4.3.

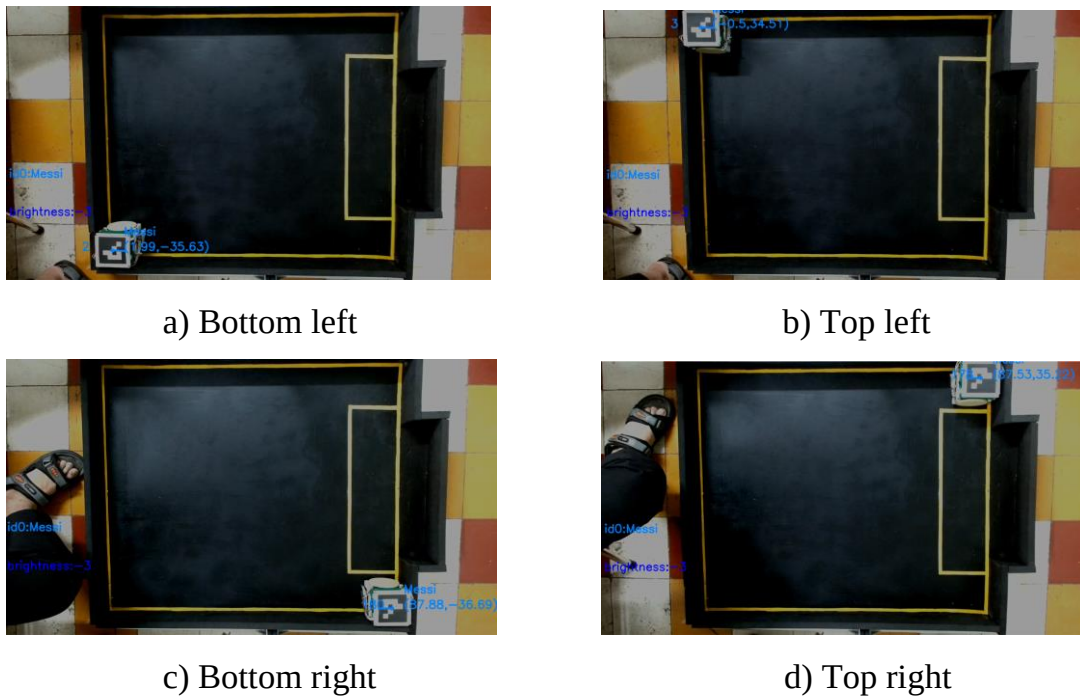


Figure 4.15 Robot positions

Table 4.3 Robot recognition error result (first method)

	Ox direction (cm)	Oy direction (cm)
Bottom left	3.01	2.63
Top left	5.00	1.51
Bottom right	2.88	3.69
Top right	2.53	2.22

The second checking method is to put the ball at the same position many times, then the coordinates of each putting cases are recorded in order to calculate the average errors.

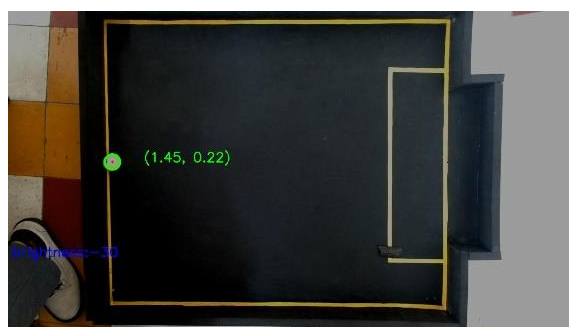


Figure 4.16 Chosen testing location

The soccer field is covered by walls, so if the chosen original point is selected to put the ball on, the real coordinate of the ball would be (2.15,0). Figure 4.16 demonstrates the chosen location, which is the chosen original on soccer field, to test the accuracy of the ball position in the second method. Finally, all data are recorded in table 4.4.

Table 4.4 Ball recognition error result (second method)

Case	Ox	Oy
1	1.85	-0.11
2	1.94	0.03
3	1.45	0.22
4	2.20	-0.31
5	2.10	0.28
Average	$\bar{x} = 1.91$	$\bar{y} = 0.11$

From data in table 3.10, the standard deviation can be determined

$$\sigma_x = \sqrt{\frac{1}{4} \sum_{i=1}^5 (x_i - \bar{x})^2} \approx 0.29(cm)$$

$$\sigma_y = \sqrt{\frac{1}{4} \sum_{i=1}^5 (y_i - \bar{y})^2} \approx 0.26(cm)$$

Then the standard error can be calculated:

$$\sigma_{\bar{x}} = \frac{\sigma_x}{\sqrt{5}} \approx 0.13(cm)$$

$$\sigma_{\bar{y}} = \frac{\sigma_y}{\sqrt{5}} \approx 0.12(cm)$$

Based on 3σ standard with 97% of reliability, the errors are quite small and can be acceptable. The sources of these small errors might come from:

- Camera location is not exactly at the center of the soccer field
- Light noise adversely affects to the golf ball color
- The height of the robot might reduce the accuracy of its real-life position

4.8 Conclusion

Basically, the computer vision module has satisfied the initial objectives. However, there are limitations such as:

- Light noise problems, which adversely affect to ball detection, cannot be

Chapter 4: Computer vision

handled thoroughly.

- The ball and the floor color are the same, which leads to the color detection noises.
- The objects coordinates are quite different from the realistic position.
- The set-up process is not optimized, leads to consuming time problem.

CHAPTER 5: CONTROL ALGORITHM

In this chapter, the control algorithm working principle is introduced and explained clearly. With the challenge of the soccer robot project, the control algorithm must solve 4 main challenging problems: storing data, strategy for the robot to shoot a ball, communication of PC and the soccer robot, controller design of the soccer robot. The project is the combination of knowledge of robotics, computer science, network communication, ... To achieve the goal of the project, the criteria to design the control algorithm is effectively needed.

5.1 The criteria to design the control algorithm

- The pose of the robot and ball position must be accurately stored
- The soccer robot can successfully kick the ball into the goal in many different situations
- The communication between PC and robot must be accurate and fast in short period of time
- The robot works automatically and there is no malfunction in testing process.

5.2 General workflow

As mentioned, soccer robot project is the project gathers many manipulations and processes. The need of creating a workflow is very necessary to easy control and design the control algorithm for the project.

Figure 5.1 illustrates the general workflow of the soccer robot project.

Summary of the process:

- Set up:
 - Let the robot and PC to connect the same Wi-Fi domain.
 - Put the robot and a ball lay stationary on the field.
- Perform:
 - Turn on computer vision program, all the necessary coordinates are recorded and written in JSON file.
 - The path planning program (A Star Algorithm) opens the JSON file to get suitable data, then use them to create a virtual path, which is a set of point for the robot will follow to shoot a ball. The path can be showed by the simulation and written into the JSON file.

- The socket program encodes the suitable value in JSON file and sends it for the robot via Wi-Fi.
- If the micro controller ESP32 successfully access the Wi-Fi, in this case the robot can receive the messages from PC host and then perform the task.

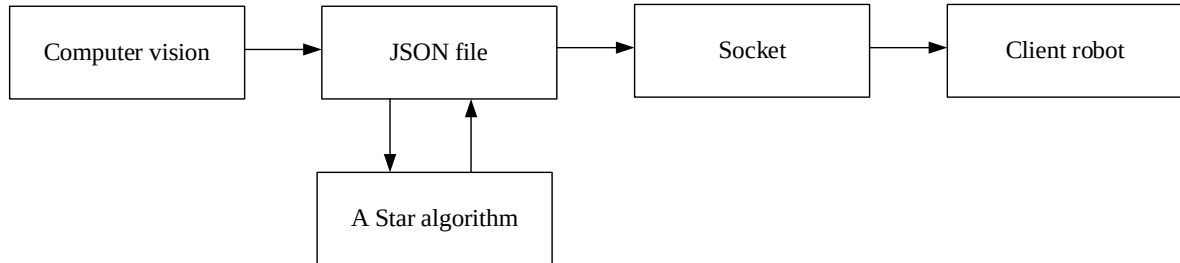


Figure 5.1 General workflow of the soccer robot project

5.3 JSON

JSON stands for JavaScript Object Notation, which is a text format for storing and transporting data. A JSON file stores data in key-value pairs and arrays, the software it was made for then accesses the data. JSON allows developers to store various data types as human-readable code, with the keys serving as names and the values containing related data. The most significant point to remember about JSON files is that they are relatively simple to use. This point creates many benefits like compatibility across platforms and a wide range of performance perks.[16]

JSON syntax is derived from JavaScript object notation syntax:

- Data is in key/value pairs
- Data is separated by commas
- Curly braces hold objects
- Square brackets hold arrays

For examples, the format of the JSON file:

```
{"ball_x": 26.94, "ball_y": 27.59, "robot_x": 13.96, "robot_y": -16.91, "robot_theta": 91.0, "move_point_list": [[50, 10], [45, 10], [40, 10], [35, 10], [30, 15], [25, 15], [20, 15], [15, 10], [10, 5], [10, 0], [10.91, 1.6], [23.91, 1.37]]}
```

With:

- The keys: ball_x, ball_y, robot_x, ...
- The value: the value of the key with data type is string, number, array, boolean, ...

5.4 Strategy to shoot a ball

5.4.1 Path planning

Path planning can only be applied when a map of the environment is known. Only the robots that are capable of SLAM (Simultaneous localization and mapping) can therefore use optimum coverage path planning approaches in order to achieve systematic covering of the entire free space. In the figure 5.2, there are various ways to use SLAM and that it's software and even components can considerably change between interactions. SLAM is first and foremost a way to interpret information, meaning that the technology and calculations employed to achieve this can vary.

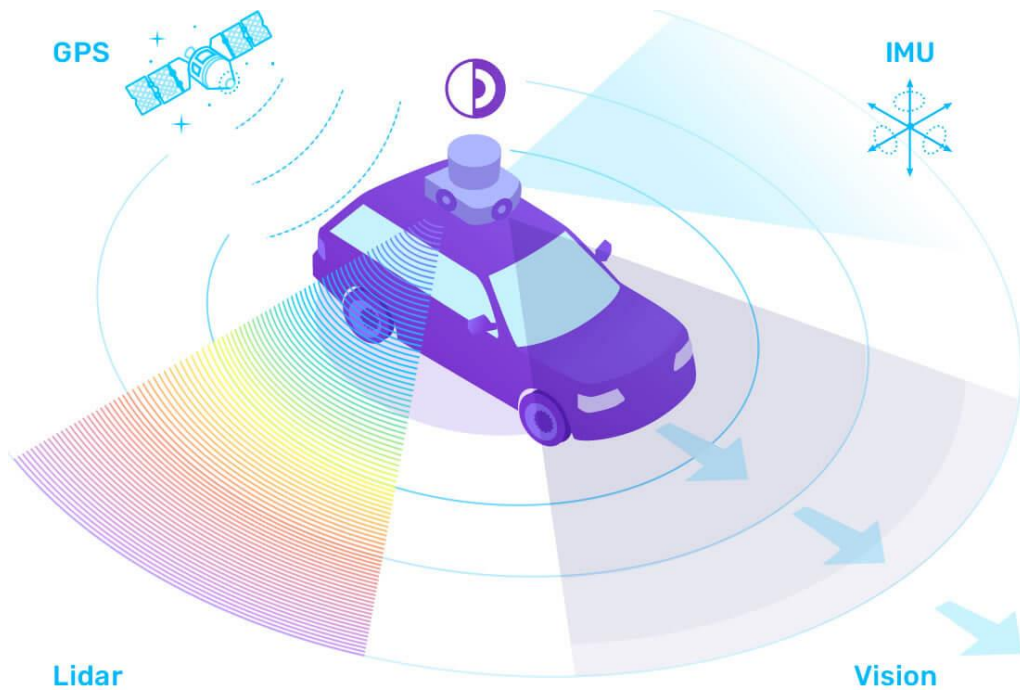


Figure 5.2 Types of SLAM[17]

Path planning is the most important issue in vehicle navigation. It is defined as finding a geometrical path from the current location of the vehicle to a target location such that it avoids obstacles. This path must be navigable by the vehicle and optimal in terms of at least one variable so that it can be considered a suitable path. For different target distance situations, the smoothest path, the shortest path, or the path along which the vehicle can move with the highest speed can become the most important path. In other words, the optimal path is determined concerning these characteristics. The figure 5.3 simulates the process of creating a path for the robot to shoot a ball into the net with the start point is robot's position and goal point is delivering a ball right into the middle of the net.

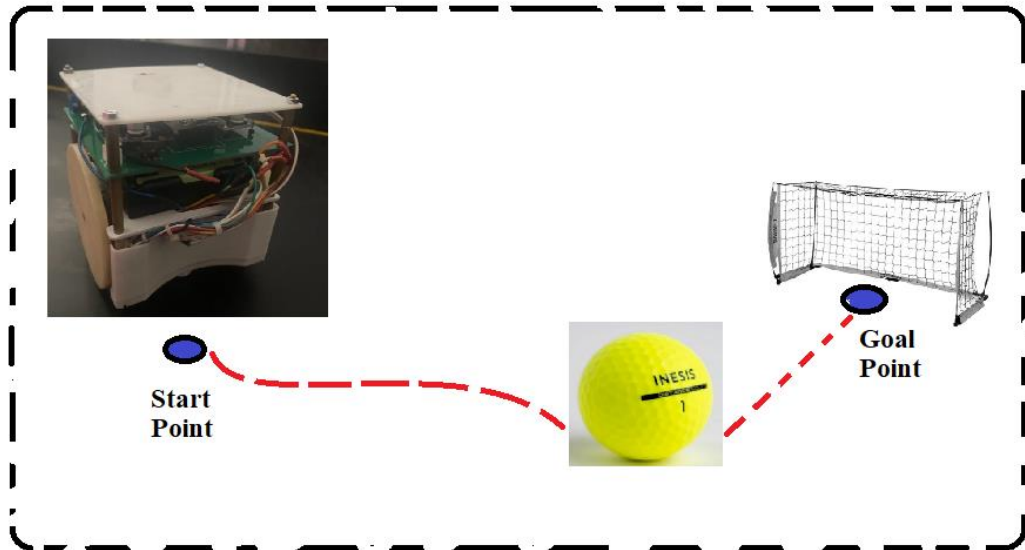


Figure 5.3 Path planning objectives of the soccer robot project

5.4.2 A* algorithm

As mentioned before, the main objective of the path planning algorithm is creating a short and effective path that robot can go through this path to shoot successfully a ball the net. With the huge contribution of the computer science, there are many path planning algorithms. To utilize the shortest path and avoid obstacle while running the project, A Star Algorithm was carefully selected.

A* (pronounced "A-star") is a graph traversal and path search algorithm, which is used in many fields of computer science due to its completeness, optimality, and optimal efficiency. It is a searching algorithm that is used to find the shortest path between an initial and a final point. It searches for shorter paths first, thus making it an optimal and complete algorithm. An optimal algorithm will find the least cost outcome for a problem, while a complete algorithm finds all the possible outcomes of a problem.

Another aspect that makes A* so powerful is the use of weighted graphs in its implementation. A weighted graph uses numbers to represent the cost of taking each path or course of action. This means that the algorithms can take the path with the least cost and find the best route in terms of distance and time.

A* algorithm works based on 3 parameters:

- $g(n)$: The movement cost to move from the starting point to a given square on the grid, following the path generated to get there

Chapter 5: Control Algorithm

- $h(n)$: The estimated movement cost to move from that given square on the grid to the final destination. This is often referred to as the heuristic, which is nothing but a kind of smart guess.
- $f(n)$: The actual cost of traversal from the initial state to the goal state.

$$f(n) = g(n) + h(n)$$

What A* Search Algorithm does is that at each step it picks the node according to a value 'f' which is a parameter equal to the sum of two other parameters – 'g' and 'h'. At each step it picks the node/cell having the lowest 'f', and process that node/cell.

The working steps of the A Star Algorithm [18]:

- Initialize the open list
- Initialize the closed list and put the starting node on the open list (can leave its f at zero)
- While the open list is not empty
 - find the node with the least f on the open list, call it "q"
 - pop q off the open list
 - generate q's 8 successors and set their parents to q
 - for each successor
 - i. if successor is the goal, stop search
 - ii. else, compute both g and h for successor
 - $\text{successor.g} = \text{q.g} + \text{distance between and q}$
 - $\text{successor.h} = \text{distance from goal to successor}$ (This can be done using many ways, there are three heuristics- Manhattan, Diagonal and Euclidean Heuristics)
 - $\text{successor.f} = \text{successor.g} + \text{successor.h}$
 - iii. if a node with the same position as successor is in the OPEN list which has a lower f than successor, skip this successor
 - iv. if a node with the same position as successor is in the CLOSED list which has a lower f than successor, skip this successor.
 - Otherwise, add the node to the open list
- End (for loop)
- Push q on the closed list
- End (while loop)

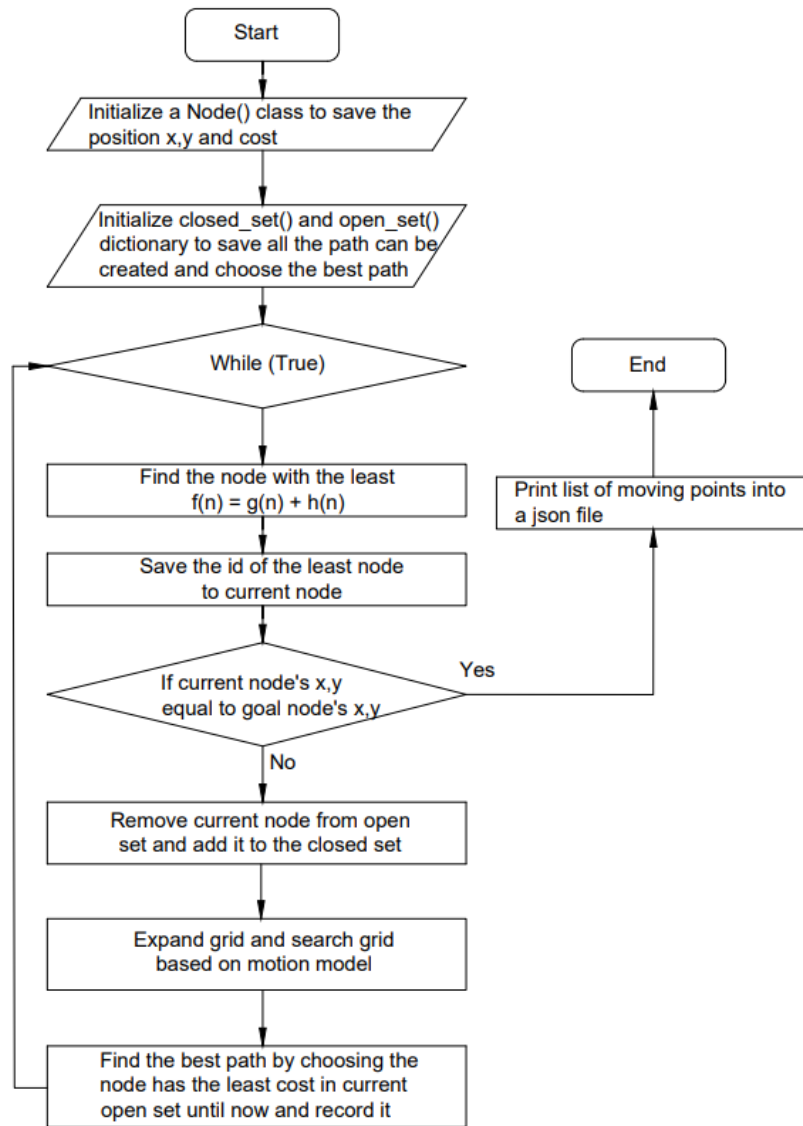


Figure 5.4 A* flowchart

Figure 5.4 describes the A* flowchart basically. With the support of JSON file, the list of points robot needed to go through has easily save in the key – value format, for example: The A* generate a list of n points so the JSON file will generate “move_point_list” : [[x1, y1], [x2, y2], ..., [xn, yn]].

5.4.3 Heuristics

The heuristic function $h(n)$ tells A* an *estimate* of the minimum cost from any vertex n to the goal. It's important to choose a good heuristic function. There are generally three approximation heuristics to calculate h :

- Manhattan distance:

It is the sum of absolute values of differences in the goal's x and y coordinates and the current cell's x and y coordinates respectively

$$h = \text{abs}(\text{current_cell}.x - \text{goal}.x) + \text{abs}(\text{current_cell}.y - \text{goal}.y)$$

This method is used when the robot is allowed to move in 4 directions only. The figure 5.5 shows the Manhattan Distance heuristics (assume red spot as source cell and green spot as target cell).

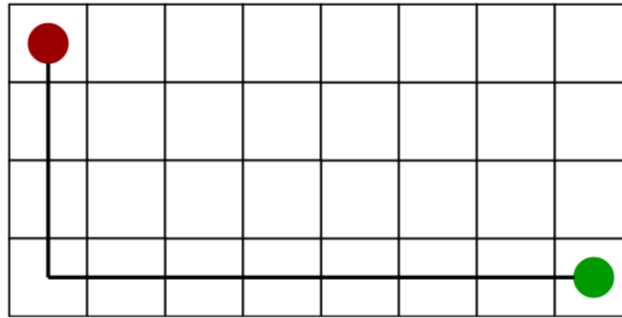


Figure 5.5 Manhattan Distance heuristics

- Diagonal distance

The maximum of absolute values of differences in the goal's x and y coordinates and the current cell's x and y coordinates respectively.

$$dx = \text{abs}(\text{current_cell}.x - \text{goal}.x)$$

$$dy = \text{abs}(\text{current_cell}.y - \text{goal}.y)$$

$$h = D \times (dx + dy) + (D2 - 2 \times D) \times \min(dx, dy)$$

where D is length of each node (usually = 1) and D2 is diagonal distance between each node (usually = $\sqrt{2}$).

This method is used when the robot is allowed to move in 8 directions. The figure 5.6 shows the Diagonal Distance heuristics (assume red spot as source cell and green spot as target cell).



Figure 5.6 Diagonal Distance heuristics

- Euclidean distance

It is the distance between the current cell and the goal cell using the distance formula

$$h = \text{sqrt}((\text{current_cell}.x - \text{goal}.x)^2 + (\text{current_cell}.y - \text{goal}.y)^2)$$

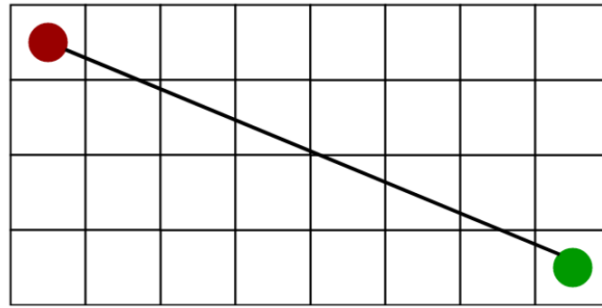


Figure 5.7 Euclidean Distance heuristics

This method is used when the robot is allowed to move in any directions. The figure 5.7 shows the Euclidean Distance heuristics (assume red spot as source cell and green spot as target cell).

Conclusion: Choose Diagonal Distance heuristics method because the robot is allowed to move in any directions and this method can easily avoid obstacle when knowing its position.

5.4.4 Evaluation

The strategy for the robot to shoot the ball is creating a line from a middle of the goal into the ball, then find the symmetry point with a length is 13 centimeters because this is the safe distance for the robot to kick a ball. Next, using A Star Algorithm to find a suitable path with a list of points to go the point that was needed to shoot a ball.

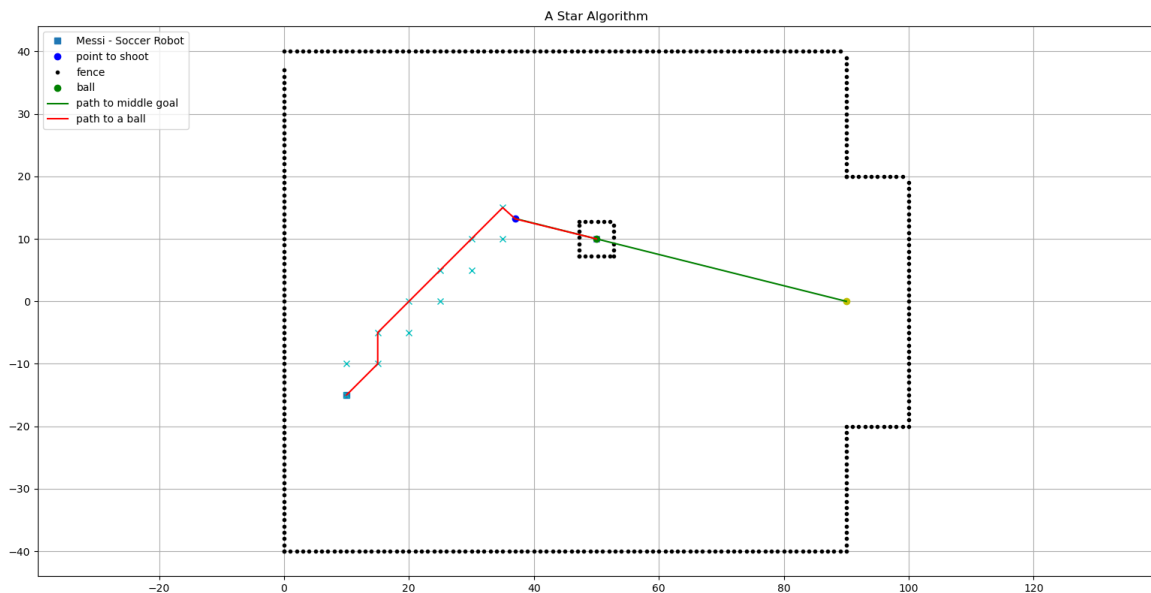


Figure 5.8 Path planning simulation

After all the necessary parameters are collected from computer vision system, the path planning file is activated. The figure 5.8 shows the simulation of finding final

path after initializing all necessary parameters. In the top left side of the figure show the detail comment:

The red spot is the robot location, green spot is the ball location, and the blue spot is the first goal that the robot should move to with the grid size is 5 centimeters. All surrounded black spots are obstacle location, might be a ball or the soccer field wall.

In this test, the format of JSON file looks like:

```
{"ball_x": 50, "ball_y": 10, "robot_x": 10, "robot_y": -15, "robot_theta": 91.0, "move_point_list": [[15, -10], [15, -5], [20, 0], [25, 5], [30, 10], [35, 15], [37.0, 13.2], [50, 10]]}
```

With A Star Algorithm, the robot can also avoid the obstacle, for example: other robots, ball, the barrier, ... An add-on dodge enemy robot function also can be created, if there is an enemy on the way as an obstacle, the moving point is going to be generated to dodge. Figure 5.9 is a dodging enemy simulation.

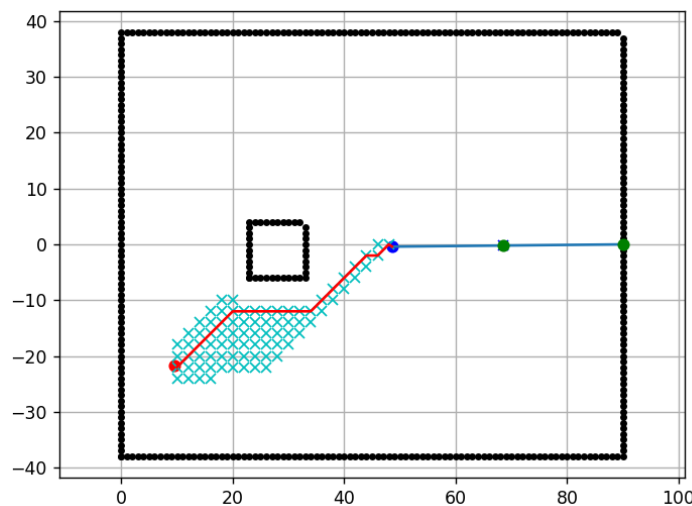


Figure 5.9 Simulation to dodge enemy

5.5 Socket network

A socket is one endpoint of a two-way communication link between two programs running on the network. The socket mechanism provides a means of inter-process communication (IPC) by establishing named contact points between which the communication take place. A socket connecting to the network is created at each end of the communication. Each socket has a specific address. This address is composed of an IP address and a port number. Socket is generally employed in client server applications. The server creates a socket, attaches it to a network port address then waits for the client to contact it. The client creates a socket and then attempts to

connect to the server socket. When the connection is established, transfer of data takes place.

Socket programming is a way of connecting two nodes on a network to communicate with each other. One socket (node) listens on a particular port at an IP, while the other socket reaches out to the other to form a connection. The server forms the listener socket while the client reaches out to the server.

5.5.1 The client-server model

The client-server model is one of the most used communication paradigms in networked systems. Clients normally communicate with one server at a time. From a server's perspective, at any point in time, it is not unusual for a server to be communicating with multiple clients. Client needs to know of the existence of and the address of the server, but the server does not need to know the address of (or even the existence of) the client prior to the connection being established. There are 2 types of communications: UDP and TCP. Table 5.1 mentions the differences of UDP and TCP protocol.

UDP (User Datagram Protocol) is a simple transport-layer protocol. The application writes a message to a UDP socket, which is then encapsulated in a UDP datagram, which is further encapsulated in an IP datagram, which is sent to the destination. There is no guarantee that a UDP will reach the destination, that the order of the datagrams will be preserved across the network or that datagrams arrive only once.

The problem of UDP is its lack of reliability: if a datagram reaches its final destination but the checksum detects an error, or if the datagram is dropped in the network, it is not automatically retransmitted.

Each UDP datagram is characterized by a length. The length of a datagram is passed to the receiving application along with the data. No connection is established between the client and the server and, for this reason, we say that UDP provides a connection-less service. It is described in RFC 768.

TCP (Transmission Control Protocol) provides a connection-oriented service, since it is based on connections between clients and servers. TCP provides reliability. When a TCP client send data to the server, it requires an acknowledgement in return. If an acknowledgement is not received, TCP automatically retransmit the data and waits for a longer period of time. We have mentioned that UDP datagrams are characterized by

a length. TCP is instead a byte-stream protocol, without any boundaries at all.

TCP is described in RFC 793, RFC 1323, RFC 2581 and RFC 3390.

Table 5.1 Difference between TCP and UDP protocol [19]

Feature	TCP	UDP
Connection status	Requires an established connection to transmit data (connection should be closed once transmission is complete)	Connectionless protocol with no requirements for opening, maintaining, or terminating a connection
Data sequencing	Able to sequence	Unable to sequence
Guaranteed delivery	Can guarantee delivery of data to the destination router	Cannot guarantee delivery of data to the destination
Retransmission of data	Retransmission of lost packets is possible	No retransmission of lost packets
Error checking	Extensive error checking and acknowledgment of data	Basic error checking mechanism using checksums
Method of transfer	Data is read as a byte stream; messages are transmitted to segment boundaries	UDP packets with defined boundaries; sent individually and checked for integrity on arrival
Optimal use	Used by HTTPS, HTTP, SMTP, POP, FPT	Video conferencing, streaming, DNS, VoIP

Conclusion: Choose TCP. Because TCP has built-in systems to check for errors and to guarantee data will be delivered in the order it was sent, making it the perfect protocol for transferring information data files. Besides, Micro Python library of ESP and Python strongly support TCP socket.

5.5.2 ESP 32 Wi-Fi station mode:

When the ESP32 is set as a Wi-Fi station, it can connect to other networks (like your router). In this scenario, the router assigns a unique IP address to your ESP board. You can communicate with the ESP using other devices (stations) that are also connected to the same network by referring to the ESP unique IP address. The figure

5.10 show ESP32 was set as a Station and connect to Wi-Fi network

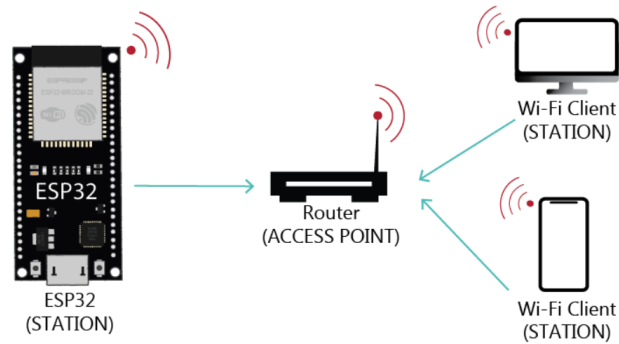


Figure 5.10 ESP 32 board act as Wi-Fi Station [20]

To connect the ESP32 to a specific Wi-Fi network, you must know its SSID and password. Additionally, that network must be within the ESP32 Wi-Fi range like normal mobile phone or laptop. The figure 5.11 show how the ESP 32 micro controller connect to the Wi-Fi and create the IP address for socket protocol.

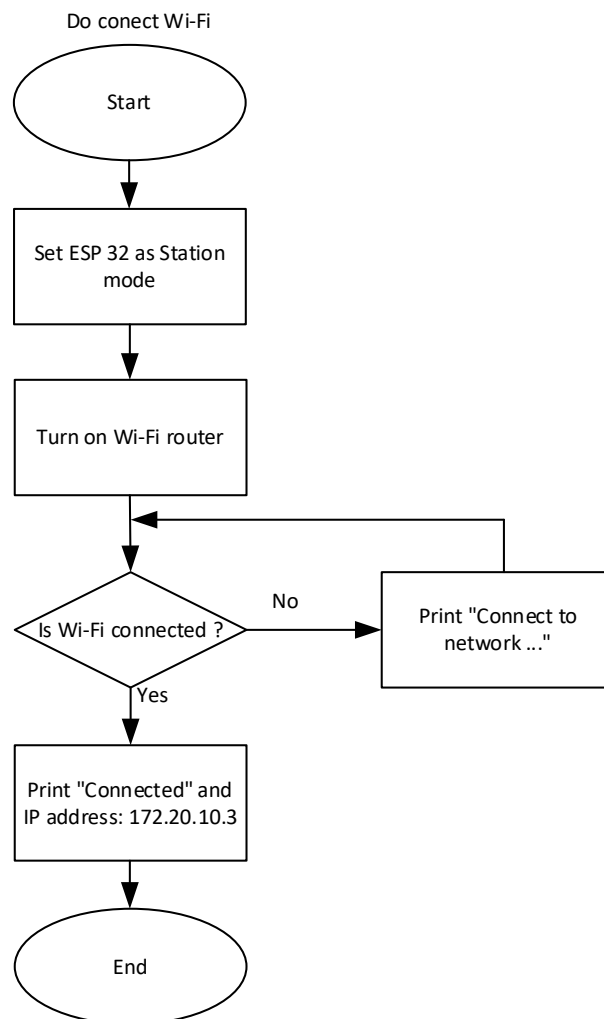


Figure 5.11 ESP 32 connect to Wi-Fi and show IP address

5.5.3 Client programing method:

Chapter 5: Control Algorithm

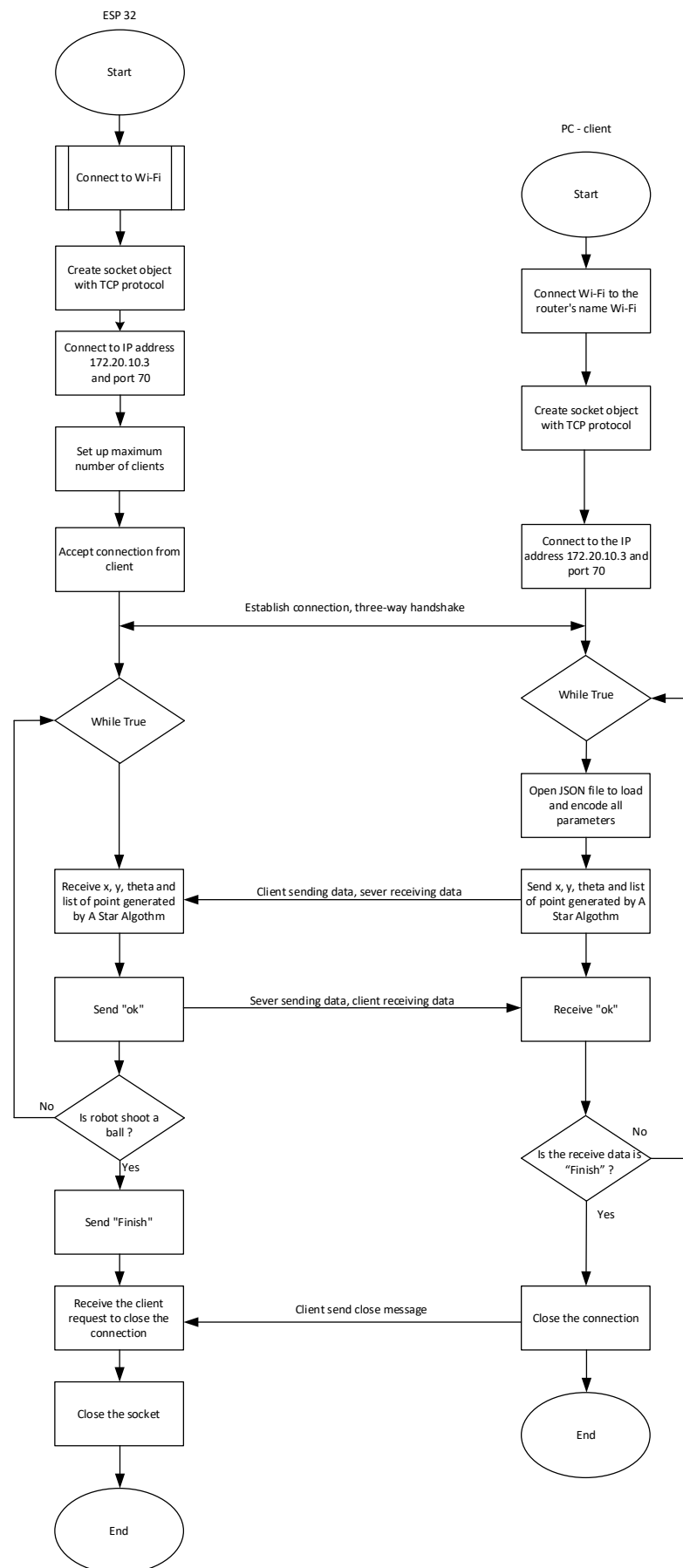


Figure 5.12 Working principle of ESP 32 and PC client program interact each other

Now need something with which a server can interact. The steps to write client program:

- First of all, make a socket object.
- Then connect to localhost on port 12345 (the port on which our server runs) and lastly, send and receive data from the server and close the connection.
- Now save this file as client.py and run it from the terminal after starting the server script

In the figure 5.12 show the working principle of ESP 32 and PC client program interact each other

5.6 Robot's main controller

Main controller program is the main part of the project, the idea of controlling the soccer robot is how to turn the robot the position that had planned from the path planning algorithm. From that idea, the duties are localizing a robot using camera to send to the robot and a PID controller need to build to make a robot move to a go location. While using computer vision, the input parameter of the robot is calculated and then sent to the robot by socket protocol. The main program uses the input parameters to drive the motor as it wants by the PID controller's sub program called "GoToGoal.py". After setting an x and y goals position, the controller should drive the robot to a goal using estimated pose calculated by the odometry. Figure 5.13 illustrates the main program flowchart.

Robot localization is the process of determining where a mobile robot is located with respect to its environment. Localization is one of the most fundamental competencies required by an autonomous robot as the knowledge of the robot's own location is an essential precursor to making decisions about future actions. First, input fixed parameter of the robot: wheel radius, wheel base.

Wheel radius = 4.3

Wheel base = 10.5

After using A Star algorithm, the list of goal point had been created and sent to the client the ESP32 by TCP – socket protocol. Then the main program initiates the position of the robot x, y and theta. The position of the robot will be updated by settling time dt (the time socket need to send position's robot messages). Same as the previous chapter mention, the robot position was handled by computer vision code and then

generate a JSON file to save x, y, theta value.

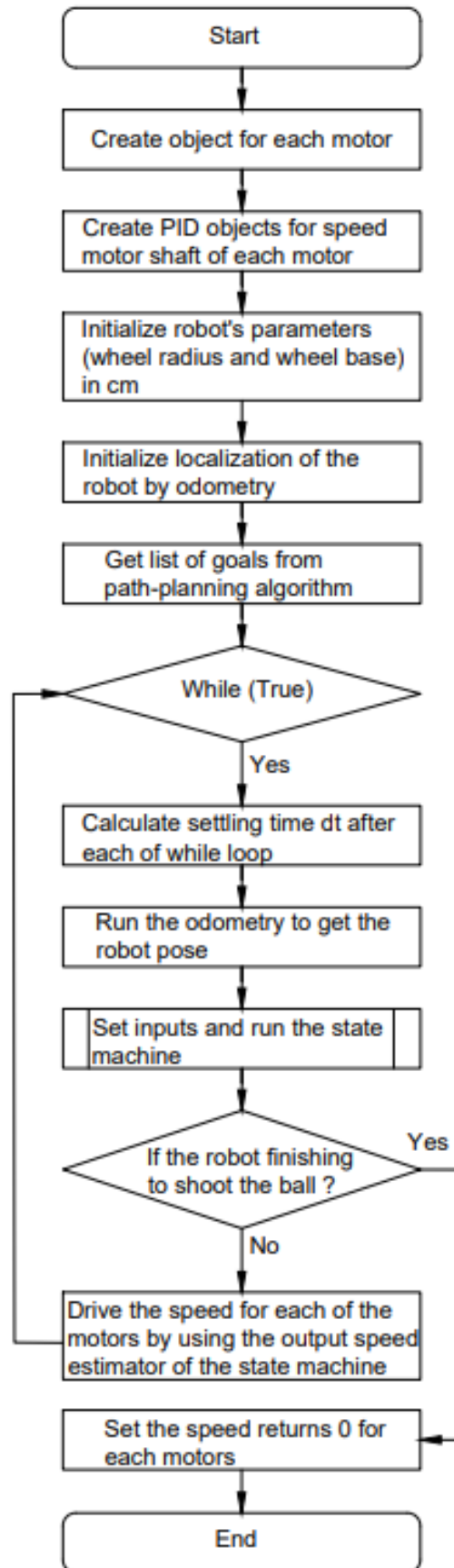


Figure 5.13 Main program flowchart

Then, the main program runs the state control to drive the robot all the state its need to go the goal position. The input of the subprogram state control is an x , y , θ of the robot, dt (settling time of the PID controller), list of the goals, wheel base and wheel's radius of the robot. Then the subprogram will run and calculate the suitable output speed for the motor to turn robot through list of goals and also kick a ball in to the net.

Finally, the main program has a duty to drive the motor using the speed output that calculated by the state control before.

5.7 Sub program localization

The sub program localization simply is a socket programming code by odometry. Odometry is the use of data from motion sensors to estimate change in position over time. It is used in robotics by some legged or wheeled robots to estimate their position relative to a starting location. This method is sensitive to errors due to the integration of velocity measurements over time to give position estimates. Rapid and accurate data collection, instrument calibration, and processing are required in most cases for odometry to be used effectively. In this soccer robot project, the global camera is a sensor to estimate the change of robot's motion. Socket programming is a way of connecting two nodes on a network to communicate with each other. One socket(node) listens on a particular port at an IP, while the other socket reaches out to the other to form a connection. The server forms the listener socket while the client reaches out to the server. The client is the robot which listen the message from the computer – the server. The computer has to send all the input parameter that robot needs: x , y , θ and list of goals. So that settling time dt is the time between 2 times the robot get its position by the server.

5.8 Sub program state control

The state machine is the main controller of the robot. The principle the state control machine is establish a library to contain 4 main states as figure 5.14.

In progress, the robot has 4 main states: “InitSt”, “GoToGoalSt”, “AtTheGoalSt”, “ShootingSt”. Each of the state has a class update the input and the output, each of class has entry and run function.

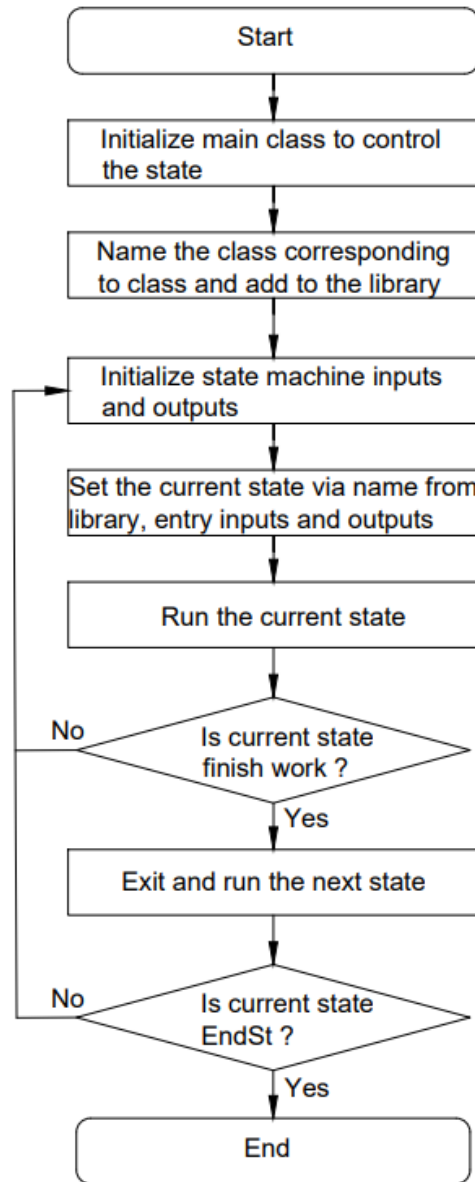


Figure 5.14 The flowchart of state machine

“InitSt”: doesn’t do anything just get ready to change to GoToGoalSt.

“GotoGoalSt”: where the robot run the PID controller, after having an array of goals is where the robot need to go. In the entry function of the state, the robot get the next goal and update goal every time the robot jump in to this function. In the run function, where PID controller run. The input of the PID controller are goal (x, y), the odometry of the robot (x, y, theta) and the settling time dt. The output is actually the target angular velocity ω . The PID controller is actually controlling how fast the robot turns, to change this to motor commands, the code needs to transform uni model to differential model.

$$\omega = K_p \cdot e + K_I \cdot \int e dt + K_D \cdot \frac{de}{dt}$$

Chapter 5: Control Algorithm

In which:

- $K_P = 1.5, K_I = 0.01, K_D = 0.5$ are PID controller parameters
- $e = \theta_{goal} - \theta$ is the error between the goal theta and real life robot's theta
- $\theta_{goal} = \arctan$ of u_y and u_x with $u_y = y_{goal} - y_{robot}$; $u_x = x_{goal} - x_{robot}$
- $dt \approx 0.1s$ is sampling time of state control machine

After PID to get the target angular velocity, so now all the things robot got are constant velocity and target theta. Let's using "Unicycle" model, which is highly useful to control how robot run and turn:

$$\begin{cases} \dot{x} = v \cos \theta \\ \dot{y} = v \sin \theta \\ \dot{\theta} = \omega \end{cases}$$

Implement this model:

$$\begin{cases} \dot{x} = \frac{R}{2}(v_r + v_l) \cos \theta \\ \dot{y} = \frac{R}{2}(v_r + v_l) \sin \theta \\ \dot{\theta} = \frac{R}{L}(v_r - v_l) \end{cases}$$

$$\rightarrow v = \frac{R}{2}(v_r + v_l) \Rightarrow \frac{2v}{R} = v_r + v_l$$

$$\rightarrow \omega = \frac{R}{L}(v_r - v_l) \Rightarrow \frac{\omega L}{R} = v_r - v_l$$

In the end, the designed velocity of each wheel:

$$v_r = \frac{2v + \omega L}{2R}$$

$$v_l = \frac{2v - \omega L}{2R}$$

In this design, v is the speed of the robot. If robot need to rotate from its middle point, let choose $v = 0$. And if in normal state choosing v to a positive constant value because the project needs to normalize the speed and maximum speed output so the robot will go as fast as it can in term of speed.

"AtTheGoalSt": this state is just zeroing out my output motor commands.

"ShootingSt": this state is just accelerating the speed of the motor to kick a ball into a net after rotating the robot into a ball.

In the PID controller, calculate u_x and u_y which is the distance between goal and

robot in x and y direction. The theta goal is simply calculated by arc tan of u_x and u_y . The error between the goal angle and robot's angle is using for the PID. Choosing the suitable gains to calibrate the robot turn optimally.

If the robot had finished “ShootingSt”, the current state now is “EndSt”. “EndSt” resets all the output of the state machine and stop the robots to end the program.

5.9 Evaluation

To assess the whole performance of the project, first let's make a question “Is the project work?” or “Can the robot actually play soccer?”. To answer the question, the test case has created to check how the system runs and to calibrate the performance the robot.

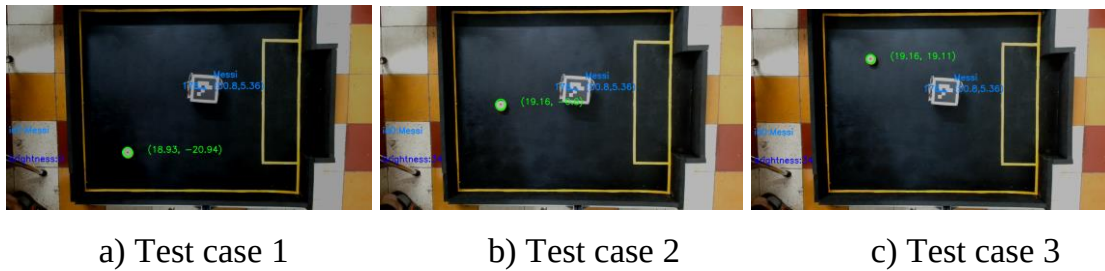


Figure 5.15 Difficult test cases

The perfect robot has an ability to kick the ball into the goal in an acceptable period of time. However, there are several positions of the ball that is difficult for the robot to operate accurately. Figure 5.15 illustrates testing positions and the table 5.2 evaluates the probability of goal successfully.

Table 5.2 Probability of goal

Case	Test case 1	Test case 2	Test case 3
1	✓	✓	✓
2	✓	✓	✓
3	✗	✓	✓
4	✓	✓	✓
5	✓	✗	✓
6	✗	✓	✗
7	✓	✓	✓
8	✓	✓	✓

9	⊗	✓	✓
10	✓	✓	✓
Probability of goal	8/10	9/10	9/10

As mentioned in this table, the probability of successfully shoot the ball into net is quite good. Now moving to evaluate the performance of the robot, let's set up 20 test cases from easy to difficult and see the performance of the robot based on: finishing time, goal status, error of goal versus middle of the goal line base on y direction. Figure 5.16 reveals the 20 test cases.

The result of the testing performance for the Messi robot soccer is indicated shortly in the table 5.3

Table 5.3 The table of the performance of the robot over 20 test cases

Test case	Finishing time (s)	Goal status	Error vs middle of the goal line on y direction (cm)
1	15.10	Goal	-14.66
2	12.12	Goal	-4.25
3	16.54	Goal	-16.31
4	10.53	Miss	NA
5	13.07	Goal	0.33
6	16.84	Goal	0.23
7	16.69	Goal	-3.47
8	24.69	Goal	2.31
9	22.24	Goal	-3.20
10	33.93	Goal	-4.38
11	23.00	Goal	0.05
12	24.83	Goal	-2.14
13	30.92	Miss	-22.69
14	35.80	Goal	5.28
15	35.33	Goal	-8.52
16	47.36	Miss	20.12
17	25.83	Goal	-13.06
18	52.91	Goal	-1.12

19	37.12	Goal	-5.83
20	43.75	Goal	-2.56
Average	27.79	85%	-3.89

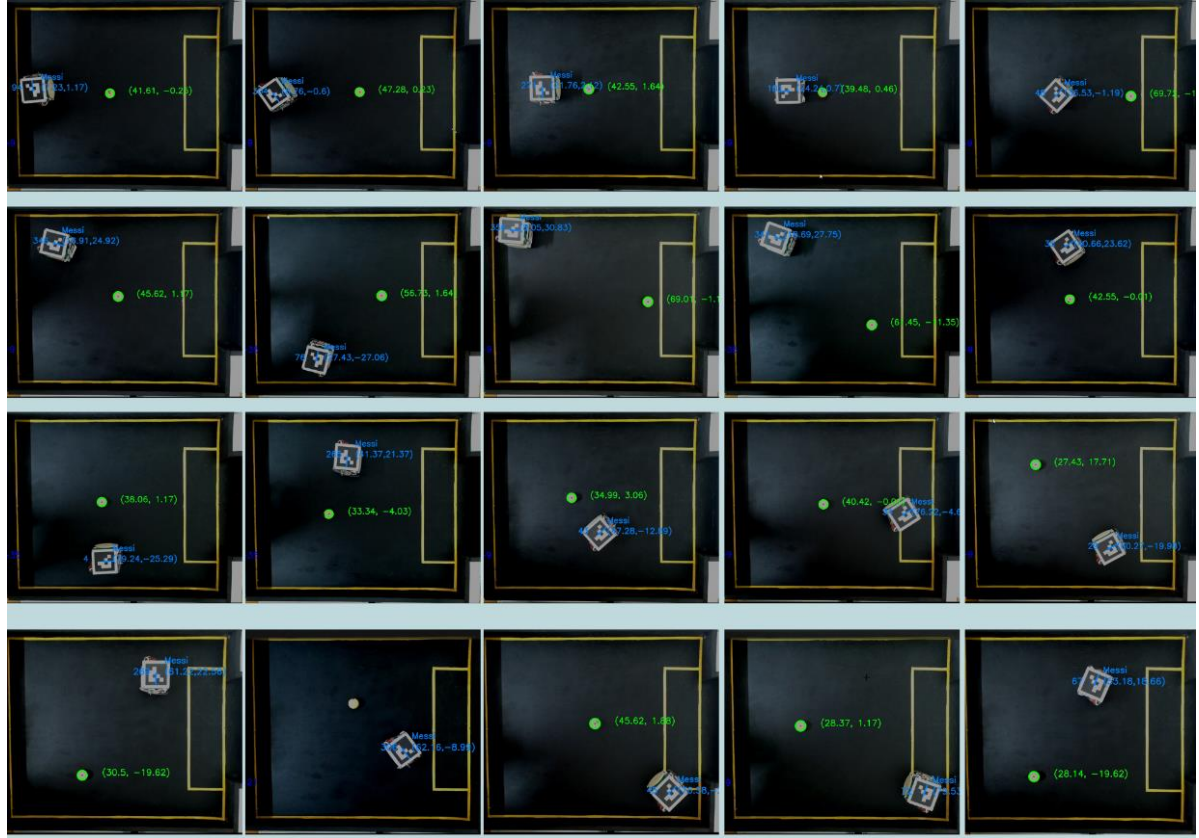


Figure 5.16 20 Test cases design to evaluate the performance of the robot (in order from left to right and up to down)

5.10 Conclusion

Over the 20 test cases, the robot already archives the target of project. Although there are some omission points in the performance like:

- The time that robot need to finishing shoot a ball is quite long.
- A few tests case the robot cannot perform well lead to miss goals
- The position of the ball when in the goal line has error up to 16 centimeters and distribute not stable throughout 20 test cases

CHAPTER 6: CONCLUSION AND SUMMARY

6.1 Summary of achievement

- Completely demonstrating the main parts of a soccer robot control system and the development situations in the foreign countries as well as in Vietnam.
- Completely designing a very small soccer robot control system. The main modules are mechanical design, electrical design, computer vision design and the algorithm control system.
- The mechanical design does not meet the technical requirement totally. In detail, the real size of the robot is larger than the initial requirement, which are 100x106x115 millimeters. However, the gear transmission still ensures the force transmission from motors to wheels.
- All modules in electrical design meet the initial requirement, which includes size, shape and function.
- Computer vision design can complete the tasks, which are recognizing ball and robot.
- The control algorithm not only helps the robot to find the most optimized path to move to a desired position, which solves the energy problem, but also to control the robot to follow to the planning path and then kick the ball with high reliability. Throughout 20 test cases, when the ball lying in goal line, the distance between the ball and the middle position of the goal is about 3.89 centimeters in negative side of Oy direction.

6.2 Summary of limitation

- The robot velocity is not fast enough; the most positive performance is when the robot velocity reach at about 0.12 m/s.
- The mechanical design is not extremely good, which leads to negative PID performance. Furthermore, gear transmission might generate loud noise when operating.
- If the soccer field is moved accidentally when the computer vision module is working, it is suggested to restart vision operation to recalibrate the coordinates.

6.3 Evolution method suggestion

- Design the mechanical system more accurate, which leads to better kicking ball performance by utilizing PID controller.

Chapter 6: Conclusion and Summary

- To increase the reliability of coordinates received from computer vision module, two cameras can be used to counter the robot height problem.
- To reduce the period of processing time, some methods can be applied such as taking advantages from more modern hardware such as camera with high fps.

References

- [1] *A Brief History of Robocup*, robocup.org,
https://www.robocup.org/a_brief_history_of_robocup
- [2] *Nao soccer robots will meet human rivals in 2050*, FayerWayer, 2019
- [3] Sven Behnke, *Playing Soccer with Humanoid Robots*, Ki – Zeitschrift Kunstliche Intelligenz, 03/2006, p.51-56
- [4] RoboCup 2019, Sydney, Australia, <https://nimbroweb.net/Humanoid/>
- [5] RoboCup Technical Committee, *RoboCup Standard Platform League (NAO) Rule Book*, robocup.org, May 16th 2022
- [6] RoboCup Standard Platform League, robocup.org, <https://spl.robocup.org/>
- [7] Robin Soetens, Bernardo Cunha, Tutorial: Requirements for RoboCup MSL, April 2016
- [8] RoboCup Junior Soccer Technical Committee, *RoboCup Junior Soccer - Rules 2018*, RoboCup.org, 2018
- [9] Carlos Henrique Caloete Pena, *An Analysis of Reinforcement Learning applied to Coach task in IEEE very Small Size Soccer*, ResearchGate, November 2020
- [10] The FIRA MiroSot Robot Soccer Playground, FIRA
- [11] Tieu Binh Tran, *Study and Implementation of a Robot Soccer System based on the CDIO approach*, Faculty of Mechanical Engineering Ho Chi Minh City University of Technology, 2021
- [12] Trinh Chat, Le Van Uyen, *Tinh toan thiet ke he dan dong co khi*, Nha xuất bản giao duc, sixth edition
- [13] ArtisTech, *Driving Small Motors with TB6612FNG*, Autodesk Instructables, 2020
- [14] Zerynth, DOIT Esp32 DevKit v1,
https://olddocs.zerynth.com/r2.6.2/official/board.zerynth.doit_esp32/docs/index.html
- [15] Hshop, LM2596 Buck Converter, <https://hshop.vn/products/mach-giamop-dc-lm2596-3a>
- [16] JSON.org, <https://www.json.org/json-en.html>
- [17] Dioram SLAM One, *What is SLAM ?*, Oct 8th 2021
- [18] Geeksforgeeks, *A* Search Algorithm*, May 30th 2022
- [19] Matt Cook, *TCP vs. UDP: What's the Difference ?*, Oct 24th 2017
- [20] Random nerd tutorials, *ESP32 Useful Wi-Fi Library Functions (Arduino IDE)*